

Programmation embarquée sur microcontrôleur

2^{ème} année cycle ingénieur

Valentin Egloff

2026-27



Patron de présentation par Laure Gonnord

Enseignement (ESISAR)

- Électronique numérique (1A)
- Architecture des processeurs (3App)
- Programmation embarquée sur microcontrôleur (4A)
- Numérique, sociétés et limites planétaires (4A/4App)
- Systèmes embarqués temps-réels (5A/5App)

Recherche (LCIS)

- Sécurité matérielle
- Attaques par injection de fautes et par canaux auxiliaires
- Conception matérielle (e.g. RISC-V)

Organisation du cours

Responsable du cours : Valentin Egloff
valentin.egloff@lcis.grenoble-inp.fr
Bureau D224

Organisation du cours

- 6 séances de cours de 1h45 → Slides sur chamilo, prise de note attendue
- 6 séances de TD de 1h45
- 4 séances de TP de 3h30

Modalité d'évaluation

- Contrôle continu sur certains TPs (40%)
- 1 évaluation intermédiaire (10%)
- 1 évaluation finale (50%)

Objectifs du cours : « à la fin de ce cours, je sais »

Extraire les informations pertinentes dans une documentation de 1000+ pages

Choisir une cible matérielle en lien avec l'application visée

- Identifier les périphériques requis
- Déterminer la puissance de calcul nécessaire (contrainte temps réel ?)
- Estimer le besoin en mémoires

Traduire une spécification d'application en une implémentation logicielle sur un microcontrôleur

- Construire l'architecture du programme (polling, interruption, multiplexage, etc.)
- Programmer les périphériques

"Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C" de Yifeng Zhu, 3^{ème} édition, 2017

Plan du cours

- ① Les microcontrôleurs
- ② Spécificités logicielles
- ③ Les périphériques
- ④ Bus et liaisons
- ⑤ Génie logiciel embarqué

① Les microcontrôleurs

Introduction

Un peu d'histoire

Architecture

② Spécificités logicielles

③ Les périphériques

④ Bus et liaisons

⑤ Génie logiciel embarqué

Système embarqué : Ensemble matériel et logiciel réalisant une ou plusieurs fonctions sous contraintes plus ou moins sévères ¹

Généralement, un système embarqué :

- dispose de ressources limitées (mémoire, puissance de calcul, énergie),
- doit être le moins cher possible (production de masse),
- ne doit pas consommer d'énergie inutilement (autonomie),
- exécute un logiciel dédié aux fonctionnalités spéciales (*firmware*),
- est soumis à de nombreuses contraintes dictées par l'environnement de l'application: poids, taille, énergie, temps, température, vibrations, haute tension, humidité, etc.

1. d'après Embedded France

Introduction : Définition

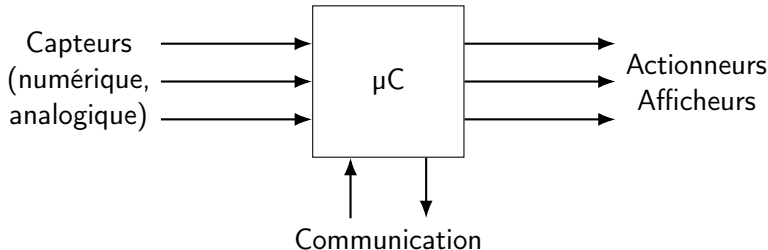
Microcontrôleur (MCU, μC): Circuit capable d'exécuter un programme et qui dispose d'interfaces avec le monde extérieur

programmable contient un microprocesseur (MPU, μP) qui exécute du code

communiquant reçoit des données de capteurs, envoie des ordres à des actionneurs (contrôle commande), communique avec d'autres μC ainsi qu'avec des serveurs distants

polyvalent est utilisé dans tous les domaines d'applications (spatial, médical, transports, électroménager, électronique grand public, etc.)

autonome à part une source d'énergie, fonctionne sans mémoire ou horloge externe



Avantages

- Faible coût (<0.01 € à ~ 10 €)
- Faible consommation (~ 10 nW en mode sommeil à 10 W)
- Forte intégration

Inconvénients

- Faible puissance de calcul (1 MHz à 1 GHz)
- Faible capacité mémoire (1 kB à ~ 100 MB)
- Contraintes thermique et mécanique fortes
- Surdimensionnement

Un peu d'histoire : Quelques cas « célèbres »

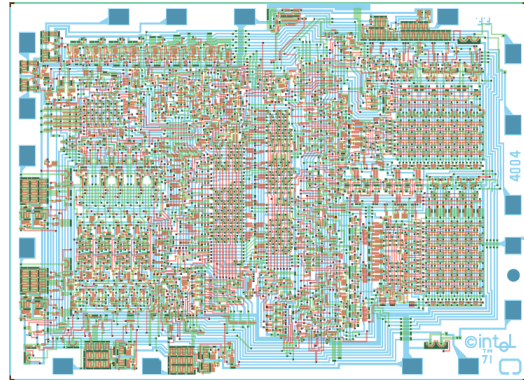
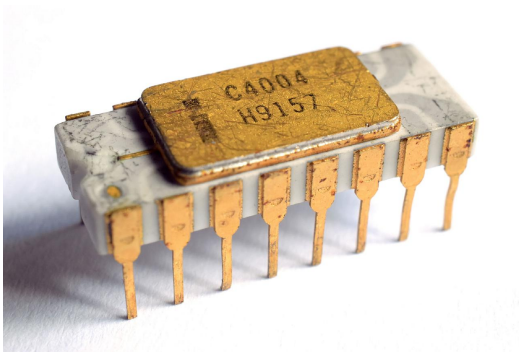
Apollo Guidance Computer

- Gère la trajectoire Terre $\rightleftharpoons$ Lune
- 72 kB de mémoire programme
- 4 kB de mémoire données
- Capteurs: centrale inertielle, mesure depuis la terre et mesure de l'équipage
- Actionneurs: propulseurs, écran
- 32 kg



Un peu d'histoire : Quelques cas « célèbres »

Premier microcontrôleur commercial : Intel 4004 (1971) 4 bits, 2300 transistors, 92kOPS, 740 kHz



Un peu d'histoire : Quelques cas « célèbres »

Ariane 5 : <https://www.ina.fr/ina-eclaire-actu/video/cac96023316/explosion-ariane-5>

- Dépassement d'entier sur le calcul de l'accélération
- Partie du code copié depuis Ariane 4
- Code non testé (toutes les simulations faites après l'accident donnent un crash)
- Tentative d'économie qui coûte plusieurs milliards de francs
- Code non nécessaire sur Ariane 5

Où trouve t-on des microcontrôleurs ?

À peu près partout:

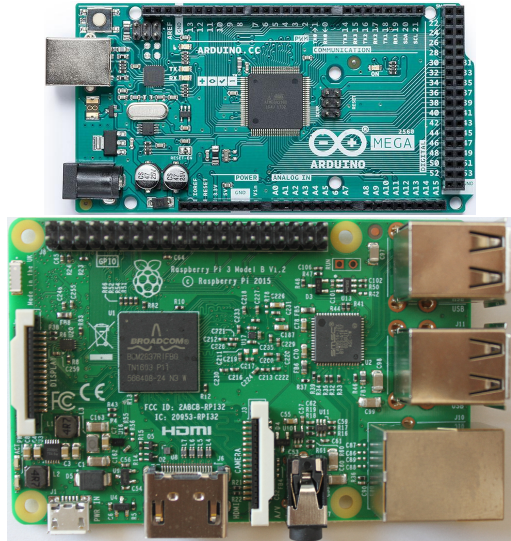
- Transports : contrôle moteur (temps réel), panneau de bord, +200 μ C dans une voiture, multimédia
- Imagerie médicale : échographie (création du signal & traitement), IRM (contrôle rotation & TS), grosso modo tous les appareils médicaux électroniques
- Environnement : station météo (relevé & télécommunication), surveillance qualité air/eau
- Bâtiments : ascenseur (contrôle moteur, carte 2G), éclairage « intelligent » (mais est-ce vraiment nécessaire ?), chaudières
- Électroménager et électronique grand public : machines à laver, cuisinière électrique, ordinateurs, téléphones, télécommandes, montres, câbles USB, etc.
- Autres : cartes à puces, cartes de transport, CB, clés & badges

Note: d'une manière générale, dès qu'il y a un affichage numérique, il y a un μ C

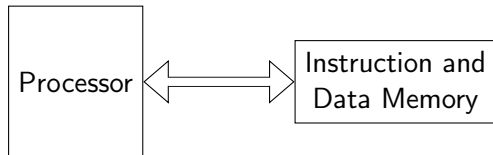
Défi : trouver un appareil sans microcontrôleur

Quelques constructeurs et familles de microcontrôleurs

- Atmel AVR (Arduino)
- Espressif ESP32 (Xtensa & RISC-V)
- Intel 8051 et 8080
- Zilog Z80 (basé sur Intel 8080, GameBoy, calculatrices)
- Microchip PIC16, PIC24, PIC32
- Texas Instruments MSP430
- Freescale (ex Motorola) 68HCxx
- ARM Cortex-M
- STM8 & STM32 (basés sur ARM)
- Beaglebone & Raspberry (cartes basées sur ARM)

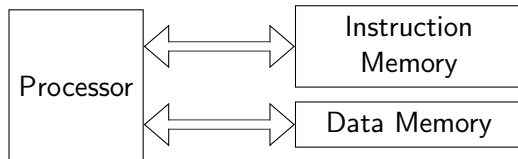


Architecture : von Neumann et Harvard



Architecture de von Neumann

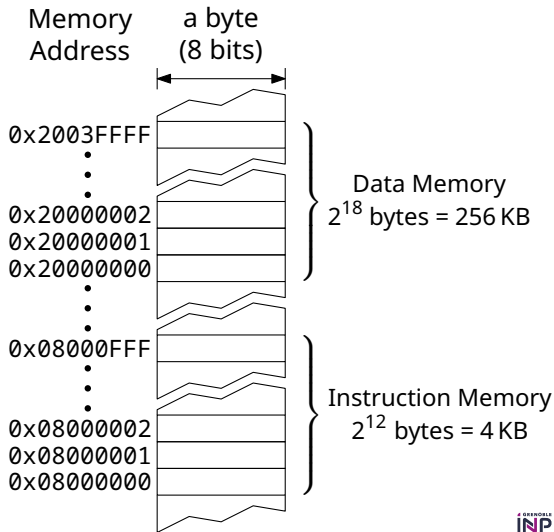
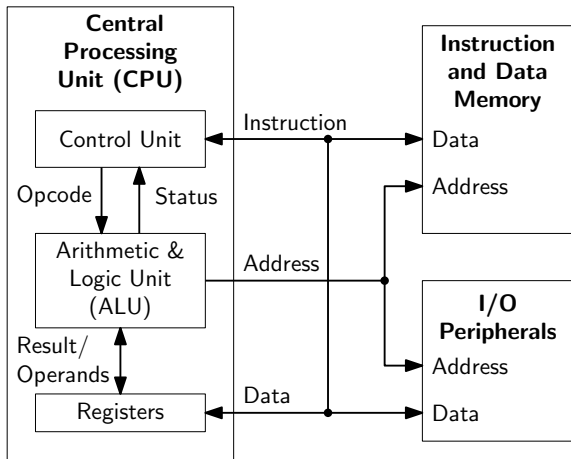
- Mémoire unifiée (un seul espace d'adressage)
- Conflit lors des load/store



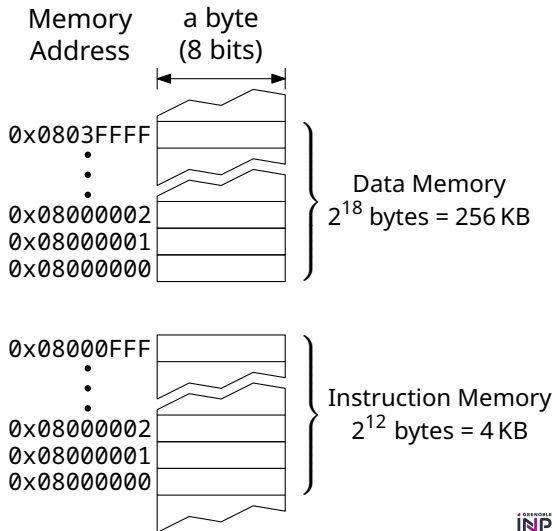
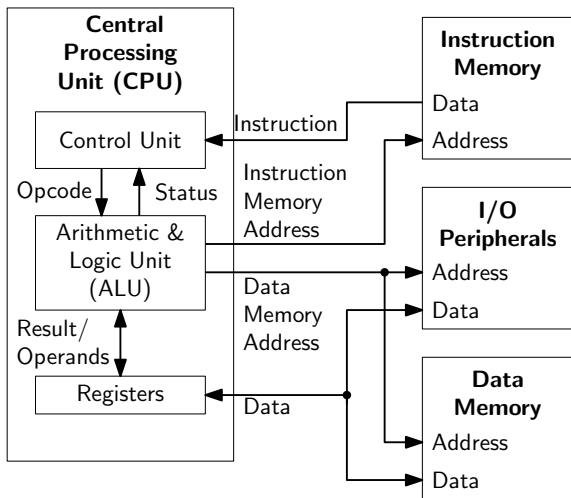
Architecture de Harvard

- Mémoires séparées
- Accès simultanés
- 2 instructions load et store différentes

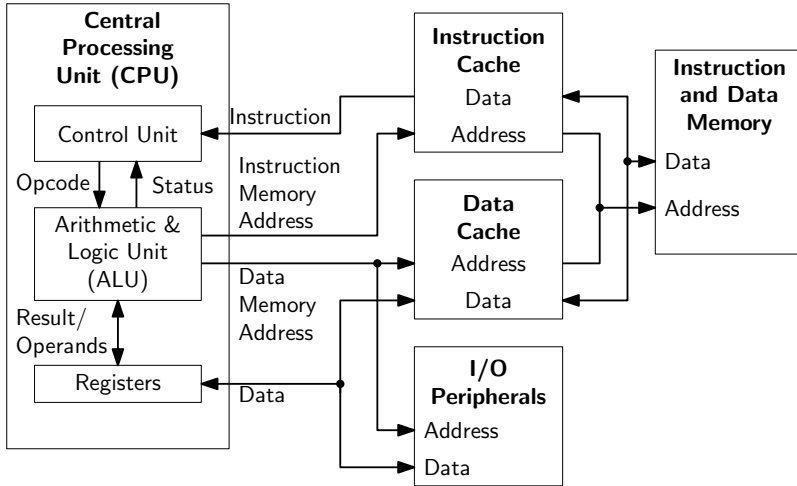
Architecture : von Neumann



Architecture : Harvard



Architecture : Harvard modifié

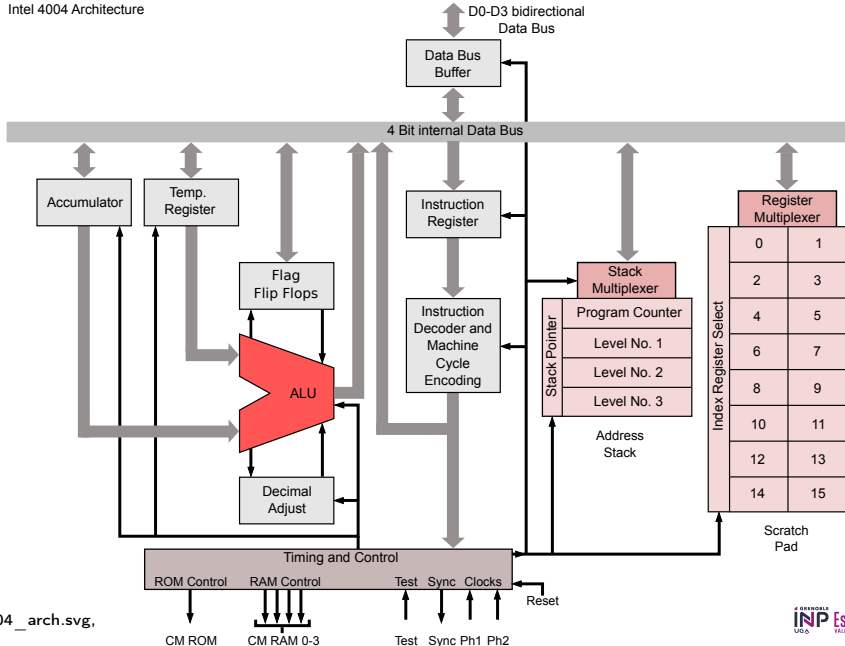


- Harvard proche CPU mais von Neumann en global
- Utilisation de caches d'instructions et de données
- Accès simultanés instruction et données (sauf conflit cache)
- Espace mémoire unifié

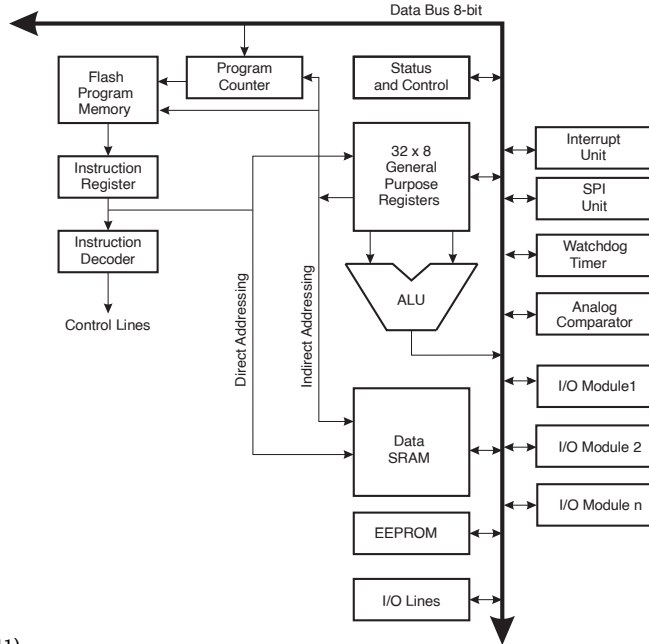
Intel 4004

- 3 cycles pour l'adresse (12 bits)
- 2 cycles pour l'instruction (1 octet)
- 3 cycles d'exécution

Quel type d'architecture ?



Atmel AVR
(Arduino)
Quel type
d'architecture ?



Le jeu d'instruction (ISA pour *Instruction Set Architecture*) est la langue parlée par un microprocesseur. Il définit le nombre de registres, leurs tailles ainsi que les instructions et l'encodage binaire. Il existe 2 grandes familles d'ISA: RISC (*Reduced Instruction Set Computer*) et CISC (*Complex Instruction Set Computer*). En général, les microcontrôleurs sont de type RISC.

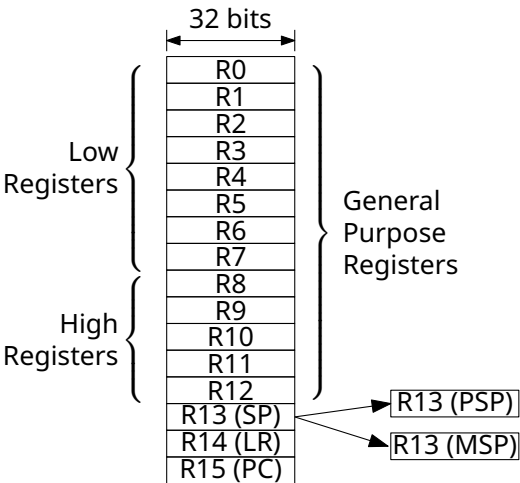
⚠ Il peut exister différentes versions d'un même ISA. En général, un code plus ancien peut tourner sur un microprocesseur plus récent, l'inverse est en général faux !

⚠ Les calculs ainsi définis sont quasi exclusivement entiers. Les calculs flottants sont rarement inclus dans les microcontrôleurs en raison de leurs coûts (énergie et surface).

De même, certaines opérations rares ou lourdes ne sont pas toujours disponibles (division et parfois multiplication).



Architecture : ISA : Exemple du Cortex-M



ADD Rd, Rs, #imm3

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	imm3			Rs			Rd		

ADD Rds, #imm8

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	Rds			imm8							

ADD.W Rd, Rs, #imm12

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	0	0	0	S		Rs		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	imm3			Rd			imm8								

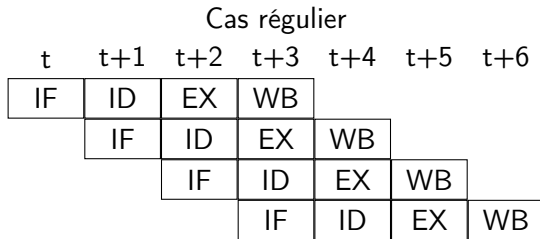
Quelques instructions spécifiques :

- Multiplication longue (64 bits) : `umull`, `smull`, 2 cycles
- Multiplication et accumulation : `mla`, 1 à 2 cycles
- Arithmétique saturée : `ssat`, `usat`, 1 cycle
- Inversion de l'ordre : `rbit`, `rev`, 1 cycle
- No Opération : `nop`

L'ISA peut définir un registre de statut qui contient des informations sur l'exécution des dernières instructions et du programme en général :

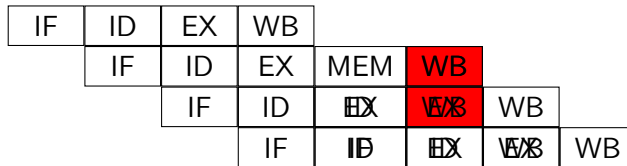
- bit de retenue **C**arry
- bit de demi-retenu **H**alf
- bit de dépassement **oV**erflow
- bit **N**égatif
- bit de nullité **Z**éro
- bit de saturation **Q**
- bit d'interruption **I**
- Exceptions et interruptions dernièrement levées

Architecture : Fonctionnement en pipeline



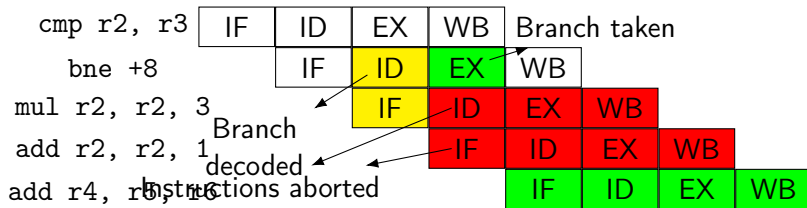
Architecture : Fonctionnement en pipeline

Cas d'un accès mémoire

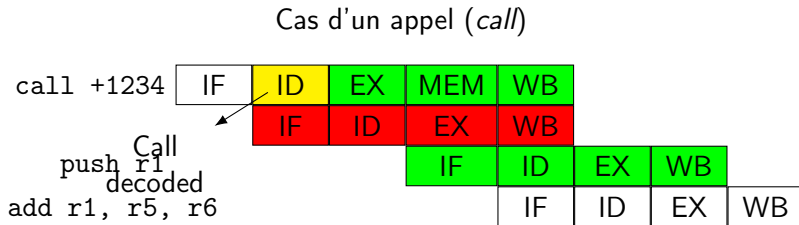


Architecture : Fonctionnement en pipeline

Cas d'un saut ou branchement



Architecture : Fonctionnement en pipeline



Les microcontrôleurs : Choix d'une cible

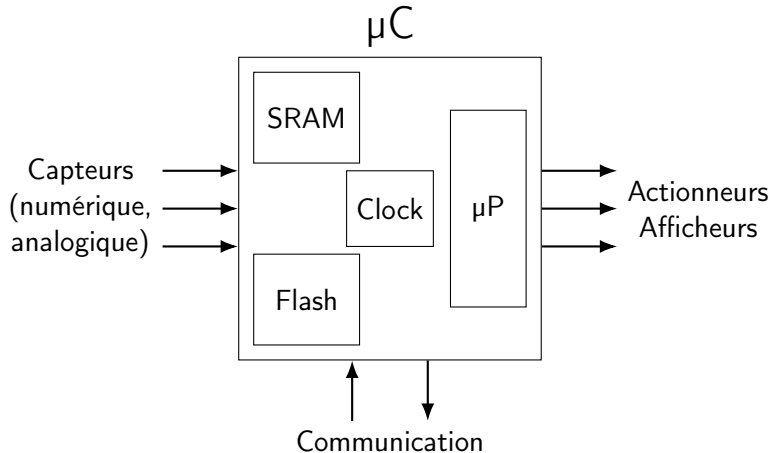
Pour bien choisir une cible matérielle en adéquation avec l'application visée, il faut :

- Lister les périphériques requis et estimer le nombre de pins requis
 - Quelles sont les données à acquérir ? (Capteurs analogiques, numériques, choix des capteurs ?) À quels débits ?
 - Quelles sont les actions à réaliser ? À quelles fréquences ?
 - Quelles communications à faire ? ( Avec les capteurs numériques, acquisition et communication sont souvent identiques) Quelles fréquences et quels débits ? Une implémentation logicielle (*bit-banging*) est-elle faisable ou acceptable ?
- Estimer la puissance de calcul nécessaire
 - Dépend des points précédents et de l'application
 - Contrainte temps réel ? (*i.e.* réagir en moins de $x \mu s$)
- Estimer la mémoire requise
- Prix et disponibilité

Les microcontrôleurs : Récapitulatif du cours

Un microcontrôleur, c'est :

- Un microprocesseur pour le calcul (avec ses spécificités architecturales)
- Des mémoires (Flash+SRAM au minimum)
- Une ou des horloges
- Des périphériques d'acquisition
- Des périphériques d'action
- Des périphériques de communication



① Les microcontrôleurs

② Spécificités logicielles

- Spécificités logicielles

- Les interruptions

- Virgule Fixe

- Bibliothèque standard

- Outils de développement

- Application Binary Interface

③ Les périphériques

④ Bus et liaisons

⑤ Génie logiciel embarqué

La programmation des systèmes embarqués se fait quasi-exclusivement en C/C++ avec un peu d'assembleur. Il existe des versions embarquées des langages suivant:

- Java avec Embedded Java
- Python avec MicroPython
- Rust avec Rust Embedded

Certains langages ont été conçu spécifiquement pour la programmation embarquée ou pour des automates:

- Ada
- Instruction List
- Structured Text
- Ladder Diagram
- Function Block Diagram

Spécificités logicielles : Différences entre code embarqué et code « local »

Code local/débarqué

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char** argv) {

    printf("Hello World from %s\n",
           argv[0]);
    char* toto = malloc(42*argc);
    // do something with toto
    free(toto);
    return 0; // yields to ?
}
```

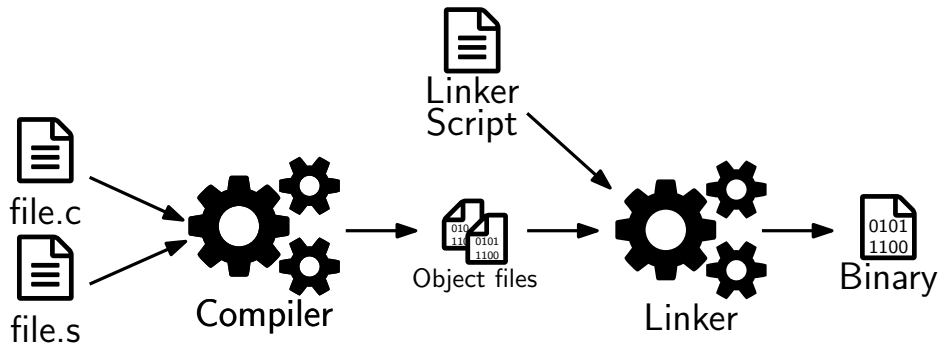
Code embarqué

```
#include <platform_specific_header.h>
#include <stdio.h>

void main(void) {
    Init_Platform(); // ??
    printf("Hello World from"
           " µC\n"); // works ?
    char toto[42];
    // do something with toto

    while(1) {
        // do something forever
    }
    return; // yields to ?
}
```

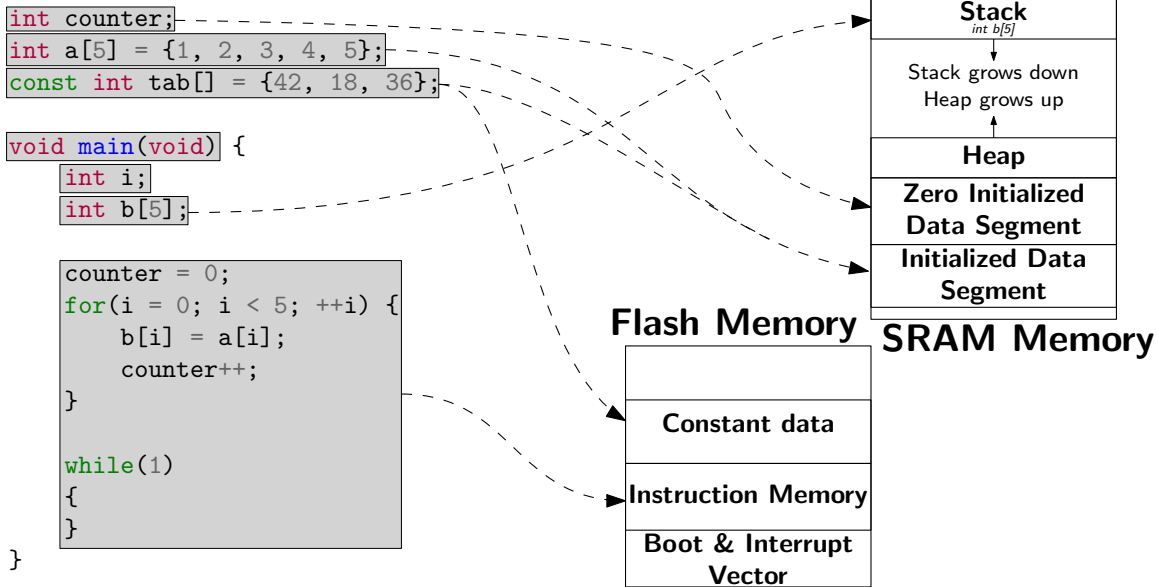
Spécificités logicielles : Processus de compilation



- `-mcpu/-mmcu/-march` pour sélectionner la cible
- `-mfpu` pour indiquer quelle FPU matérielle est disponible
- `-T file.ld` pour fournir son propre *linker script*

Processus de compilation : Exemple des Arm Cortex-M

Architecture	Core	FPU	GNU Flags
ARMV6-M	Cortex-M0	N/A	-mthumb -mcpu=cortex-m0
	Cortex-M0+	N/A	-mthumb -mcpu=cortex-m0plus
	Cortex-M1	N/A	-mthumb -mcpu=cortex-m1
ARMV7-M	Cortex-M3	N/A	-mthumb -mcpu=cortex-m3
ARMV7E-M	Cortex-M4	No	-mthumb -mcpu=cortex-m4 -mfloat-abi=soft
		Yes	-mthumb -mcpu=cortex-m4 -mfloat-abi=hard (or softfp) -mfpv4-sp-d16
	Cortex-M7	No	-mthumb -mcpu=cortex-m7 -mfloat-abi=soft
		Yes	-mthumb -mcpu=cortex-m7 -mfloat-abi=hard (or softfp) -mfpv5-sp-d16



Spécificités logicielles : Équivalence assembleur/C

Assembleur	C
<code>.section .text</code>	Déclare une section de code exécutable
<code>.global toto</code>	<code>void/int/char* toto(...); // dans le header</code>
<code>.weak toto</code>	<code>__attribute__((weak)) void toto(...); // dans le header</code>
<code>.section .data</code>	Déclare une section contenant des données modifiables
<code>.global toto</code>	<code>int/char* toto; // dans le header</code>
<code>.weak toto</code>	<code>__attribute__((weak)) int/char* toto; // dans le header</code>
<code>.section .rodata</code>	Déclare une section contenant des données non modifiables
<code>.global toto</code>	<code>const int/char* toto; // dans le header</code>
<code>.weak toto</code>	<code>__attribute__((weak)) const int/char* toto;</code>

Spécificités logicielles : Linker script

```
OUTPUT_FORMAT("elf32-avr", "elf32-avr", "elf32-avr")
OUTPUT_ARCH(avr:5)
ENTRY(Reset_Handler)    /* Specify the entry point of the program */

__FLASH_SIZE = 128K;    /* Possible to define symbols */
MEMORY /* Define memory areas */ {
    FLASH (rx): ORIGIN = 0, LENGTH = __FLASH_SIZE /* read, execute */
    RAM (rw!x): ORIGIN = 0, LENGTH = 0xffa0 /* read, write */
}

SECTIONS
{
    ...
}
```

Spécificités logicielles : Linker script

```
SECTIONS {  
    /* ISR vector table should be at beginning of FLASH */  
    .isr_vector : { ... }>FLASH  
    /* Program code and some constant data go into FLASH */  
    .text : { ... } > FLASH  
    /* Read-only data section goes into FLASH */  
    .rodata : { ... } > FLASH  
    /*Read-write initialized data section goes into RAM and FLASH */  
    .data:{ ... }> RAM AT> FLASH /*AT means both*/  
    /*Read-write uninitialized data section goes into RAM */  
    /* bss = Block Started by Symbol */  
    .bss : { ... }>RAM  
    /* Allocate a heap section in RAM */  
    ._user_heap_stack: { ... } >RAM  
}
```

Spécificités logicielles : Exemple du boot code STM32

```
.section .text
.weak Reset_Handler
Reset_Handler:
/* Copy the data segment
↪ initializers from
↪ flash to SRAM */
ldr r0, =_sdata
ldr r1, =_edata
ldr r2, =_sidata
movs r3, #0
b LoopCopyDataInit
```

```
CopyDataInit:
ldr r4, [r2, r3]
str r4, [r0, r3]
adds r3, r3, #4

LoopCopyDataInit:
adds r4, r0, r3
cmp r4, r1
bcc CopyDataInit
```

```
/* Zero fill the bss
↪ segment. */
ldr r2, =_sbss
ldr r4, =_ebss
movs r3, #0
b LoopFillZerobss
FillZerobss:
str r3, [r2]
adds r2, r2, #4
LoopFillZerobss:
cmp r2, r4
bcc FillZerobss
```

Spécificités logicielles : Exemple du boot code du STM32

```
/* Call the clock system
↳ initialization function.*/
bl SystemInit
/* Call static constructors */
bl __libc_init_array
/* Call the application's
↳ entry point.*/
bl main
bx lr

.global Default_Handler
Default_Handler:
Infinite_Loop:
b Infinite_Loop
```

```
.section .isr_vector,"a",%progbits
g_pfnVectors:
.word _estack
.word Reset_Handler
.word NMI_Handler
.word HardFault_Handler
...

.weak NMI_Handler
.thumb_set NMI_Handler,Default_Handler
.weak HardFault_Handler
.thumb_set HardFault_Handler,Default_Handler
...
```

Toujours se fier à la documentation de votre compilateur, éditeur de liens (*linker*) et cible !

- Compilateur : en général gcc, <https://gcc.gnu.org/onlinedocs/gcc/>
- Éditeur de liens : en général ld, <https://sourceware.org/binutils/docs/ld/>
- Cible : dépend du fabricant et/ou vendeur de la carte ou de la puce. Bien faire la distinction entre :
 - l'architecture (Arm, AVR, Xtensa, RISC-V, etc.) qui peut être propriétaire
 - la puce (STM32F103RBT6, ESP32-S3, ATmega328P, etc.) qui n'est que la partie CPU/MCU sans la carte
 - et la carte, souvent de développement (*Dev Board*), qui inclut la puce, un quartz, une puce UART/USB, mémoire supplémentaire, les jumpers, les connectiques, etc.

Les interruptions : Définition

Interruption : Fonction spéciale et **asynchrone** ne prenant aucun argument et ne renvoyant aucune valeur, appelée matériellement sur un évènement. Elle interrompt le programme en cours, sauvegarde les registres et le PC, traite l'évènement et rend (restauration registres et PC) la main immédiatement.

Une interruption doit :

- être la plus courte possible
- uniquement mettre à jour les données nécessaires
- utiliser des variables globales et volatiles

Exemple sur Atmel AVR (Arduino)

```
volatile long top;
ISR(INT4_vect) { // called by hardware
    ++top;
}
void main(void) {
    ...
    while(1) {
        // do something with top
    }
}
```

Les interruptions : Variables volatiles

```
long top, prevtop = -1;
void main(void) {
    while(1) {
        if(top != prevtop) {
            // top has changed
        }
    }
}
```

```
volatile long top, prevtop = -1;
void main(void) {
    while(1) {
        if(top != prevtop) {
            // top has changed
        }
    }
}
```

```
100b6: lw    a4,72(gp) # 13cf8 <top>
100ba: lw    a5,56(gp) # 13ce8 <prevtop>
100c0: bne   a4,a5,100c6 # <main+0x16>
100c4: j     100c4 # <main+0x14>
```

```
100bc: lw    a4,72(gp) # 13d08 <top>
100c0: lw    a5,56(gp) # 13cf8 <prevtop>
100c8: beq   a4,a5,100bc # <main+0xc>
```

Les interruptions : Variables volatiles

```
long top, prevtop = -1;
void main(void) {
    if(top != prevtop)
        ... // top has changed
    else
        while(1);
}
```

```
volatile long top, prevtop = -1;
void main(void) {
    while(1) {
        if(top != prevtop) {
            // top has changed
        }
    }
}
```

```
100b6: lw    a4,72(gp) # 13cf8 <top>
100ba: lw    a5,56(gp) # 13ce8 <prevtop>
100c0: bne    a4,a5,100c6 # <main+0x16>
100c4: j      100c4 # <main+0x14>
```

```
100bc: lw    a4,72(gp) # 13d08 <top>
100c0: lw    a5,56(gp) # 13cf8 <prevtop>
100c8: beq    a4,a5,100bc # <main+0xc>
```

Les interruptions : Variables volatiles

Définition : Une variable déclarée **volatile** permet de signaler au compilateur que cette variable est susceptible de changer de valeur à n'importe quel moment. Le compilateur n'optimise donc pas les accès mémoire à cette variable et est forcé de recharger sa valeur à chaque utilisation et de ranger sa valeur en mémoire à chaque modification.

```
long top;  
top++;  
top++;
```

```
volatile long top;  
top++;  
top++;
```

```
lw a4, 72(gp) # 13d08 <top>  
addi a4, a4, 2 # top += 2  
sw a4, 72(gp) # 13d08 <top>
```

```
lw a4, 72(gp) # 13d08 <top>  
addi a4, a4, 1 # top++  
sw a4, 72(gp) # 13d08 <top>  
lw a4, 72(gp) # 13d08 <top>  
addi a4, a4, 1 # top++  
sw a4, 72(gp) # 13d08 <top>
```

Les interruptions : Interrupt & Boot vector

Définition : Un microcontrôleur démarre à une adresse fixe. Les interruptions sont également à des adresses fixes connues. Ces adresses sont regroupées dans la table des interruptions (*Interrupt & Boot vector*, *Interrupt Vector Table (IVT)*, *Vectored Interrupt Controller (VIC)*). Le démarrage ou *reset* est une interruption particulière qui réinitialise le microcontrôleur.

```
g_pfnVectors:
    .word Reset_Handler ;
    ...
    .word RTC_IRQHandler ; Real Time Clock
    .word FLASH_IRQHandler
    .word EXTI0_IRQHandler ; GPIO 0
    ...
    .word TIM2_IRQHandler ; Timer 2
```

Les nombres flottants classiques permettent de représenter ces nombres mais sont gourmands en ressources de calculs. Les nombres flottants permettent également une étendue très large de 10^{-38} à 10^{38} ainsi que la gestion des infinis.

Dans l'embarqué, on manipule des données issues de capteurs et donc de l'environnement. Ces nombres sont très souvent compris dans des petites plages de valeurs (0°C à $+100^{\circ}\text{C}$ pour la température). L'utilisation de nombres flottants est donc peu pertinente.

Virgule Fixe : Définition

Dans le format de représentation en virgule fixe, la virgule est implicite et toujours au même endroit dans le nombre. Le format **DOIT** spécifier cet emplacement. Ainsi, on a un nombre binaire dont les bits de poids forts à gauche de la virgule représentent la partie entière et dont les bits de poids faibles à droite de la virgule représentent la partie fractionnaire. Si on considère un nombre au format 4.4 (4 bits entiers, 4 bits fractionnaires):

$$\begin{aligned}0111(,)0011 &= (2^2 + 2^1 + 2^0) + (2^{-3} + 2^{-4}) \\&= (4 + 2 + 1) + (0.125 + 0.0625) \\&= 7.1875_{(10)} = \frac{115}{16}\end{aligned}$$

$$\begin{aligned}0011(,)1111 &= (2^1 + 2^0) + (2^{-1} + 2^{-2} + 2^{-3} + 2^{-4}) \\&= (2 + 1) + (0.5 + 0.25 + 0.125 + 0.0625) \\&= 3.9375_{(10)} = \frac{63}{16}\end{aligned}$$

Virgule Fixe : Signe et précision



Comme on traite des nombre réels, toutes les représentations sont **signées** ! Le bit de poids fort est donc un bit de signe et on utilise la représentation en complément à 2. La partie fractionnaire est toujours positive !

$$\begin{aligned}1111(,)0011 &= (-2^3 + 2^2 + 2^1 + 2^0) + (2^{-3} + 2^{-4}) \\&= (-8 + 4 + 2 + 1) + (0.125 + 0.0625) \\&= -0.8125_{(10)} \\1100(,)0001 &= (-2^3 + 2^2) + (2^{-4}) \\&= (-8 + 4) + (0.0625) \\&= -3.9375_{(10)}\end{aligned}$$

En virgule fixe, la plus petite variation correspond à 2^{-F} avec F le nombre de bits fractionnaires. En conséquence, les nombres doivent être arrondis pour les conversions, en général vers $-\infty$.

Virgule Fixe : Conversion

La partie entière se convertit identiquement aux nombres entiers en représentation **non signée**.
Pour la partie fractionnaire, on multiplie par 2^F :

Convertir $\pi = 3.1415926535 \dots$ au format 4.12

- La partie entière vaut $E = 3$, soit 0011 sur 4 bits
- La partie fractionnaire est $F = 0.1415926535 \dots$. On calcule :
 $0.1415926535 \dots \times 2^{12} = 579.9635 \dots$
- On convertit la partie entière, donc 579 sur 12 bits **non signés**
 $579_{(10)} = 0010\ 0100\ 0011_{(2)}$

π s'écrit donc 0011(,)0010 0100 0011 en virgule fixe 4.12

Convertir $-\frac{3\pi}{2} = -4.71238898 \dots$ au format 8.8

- $E = -4_{(10)} \rightarrow 0000\ 0100_{(2)}$ sur 8 bits **non signés**
- $0.71238898 \dots \times 2^8 = 182.37 \dots$
- $182_{(10)} = 1011\ 0110_{(2)}$ sur 8 bits **non signés**

$+\frac{3\pi}{2}$ s'écrit donc 0000 0100(,)1011 0110. On fait le complément à 2 sur **l'ensemble** partie entière et partie fractionnaire

$\overline{0000\ 0100\ 1011\ 0110} + 1 = 1111\ 1011\ 0100\ 1010$
 $-\frac{3\pi}{2}$ s'écrit donc 1111 1011(,)0100 1010 en virgule fixe 8.8

Virgule Fixe : Calculs

L'addition et la soustraction s'effectuent comme pour les entiers. Pour la multiplication et la division, il faut renormaliser les nombres. En effet, on peut représenter les nombres en virgule fixe comme des fractions :

$$\begin{array}{rclcl} & 5.6250 = 90/16 & = 0101(,)1010 & & 0.0625 = 1/16 = 0000(,)0001 \\ \times & 1.9375 = 31/16 & = 0001(,)1111 & & \div 0.0625 = 1/16 = 0000(,)0001 \\ \hline 10.8984\cancel{3750} = 174/16 & = 1010\,1110\cancel{0110} & & & \hline & 1 = 16/16 = \cancel{0001}0001\,0000 & & & \end{array}$$

Si on veut avoir le même format en sortie, il faut diviser le résultat final par 16.

Si on veut avoir le même format en sortie, il faut remultiplier le résultat final par 16.

Dans le cas où les 2 nombres n'ont pas le même format, il faut mettre les 2 nombres au même format pour l'addition et la soustraction. Pour la multiplication et la division, on peut faire les calculs directement et renormaliser vers l'un des 2 formats en sortie.

Virgule Fixe : Avantages et inconvénients

Avantages

- Calculs identiques aux entiers
- Utilisation des mêmes opérateurs matériels
- Consomme moins d'énergie
- Un peu plus performant que virgule flottante matérielle

Inconvénients

- Précision limitée
- Amplitude très limitée pour le calcul scientifique
- Impossible de représenter certaines valeurs spéciales (∞ , calcul impossible, etc.)

- `#include <stdxxx.h>` ✓  incomplet
Certains en-têtes peuvent ne pas exister ou être incomplets
- `printf("Hello World!");` ✓  incomplet
Compile, mais n'affiche rien
- `malloc/free` et `new/delete` ✗
Compile, mais qui gère la mémoire sans système d'exploitation ?

Chaque plateforme fournit en général son propre environnement de développement (IDE : *Integrated Development Environment*). Celui-ci vous permet d'écrire, de compiler et déboguer facilement votre code pour votre cible embarquée.

Quelques IDEs et plateformes associées:

- Arduino IDE pour Atmel
- ESP-IDF pour Espressif
- STM32CubeIDE pour STM32 (Arm)
- MPLab IDE pour PIC
- Keil Microchip pour diverses cibles
- Visual Studio Code avec plugins pour quasiment toutes les cibles
- N'hésitez pas à utiliser l'auto-complétion !
- N'hésitez pas à bien configurer votre IDE pour avoir **VOS** raccourcis

Application Binary Interface : Définition

ABI : interface de bas niveau entre les applications et le système d'exploitation, entre une application et une bibliothèque ou bien entre différentes parties d'une même application ²

Globalement, l'ABI définit :

- le rôle des registres et la responsabilité de leur intégrité (appelant ou appelé)
- l'ordre des variables dans la pile
- les conventions d'appels, *i.e.* les registres dans lesquels sont passés les arguments ainsi que l'ordre des arguments dans la pile
- les conventions d'appels systèmes (cas avec un système d'exploitation temps réel)

L'ISA peut imposer certains choix, notamment le pointeur de pile, par certaines instructions optimisées qui peuvent être définies.

2. d'après Wikipedia

Application Binary Interface : Exemple

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5	t0	Temporary/alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

Application Binary Interface : Pourquoi c'est important ?

Dans certains cas très particuliers (contrainte temps réel, espace mémoire réduit ou nombre d'instructions limité), il est nécessaire d'écrire une partie du code en assembleur. Le programmeur a alors différents choix :

- Écrire du code assembleur *inline* avec le code C
 - Compilateur gère l'ABI pour vous
 - Parfois pas assez optimisé
- Écrire du code dans un fichier assembleur (extension `.s`) et appeler la fonction depuis le C
 - Plus complexe à mettre en œuvre → Vous devez gérer l'ABI
 - Gestion du microcontrôleur au cycle près

Les différences entre code débarqué et code embarqué:

- Utilisation de tableaux statiques
- Utilisation de variables globales et volatiles
- Utilisation de la virgule fixe préférée
- Utilisation d'interruptions
- Une boucle infinie dans le *main*
- Des en-têtes spécifiques à chaque cible
- Bibliothèque standard partiellement disponible
- Un environnement de développement (éditeur, compilateur, débogueur) spécifique à chaque cible

Plan

① Les microcontrôleurs

② Spécificités logicielles

③ Les périphériques

- GPIO

- Programmation des périphériques

- Interruptions

- Timers

- CAN

- CNA

- DMA

- Horloge & Mode économie d'énergie

- Autres périphériques particuliers

④ Bus et liaisons

⑤ Génie logiciel embarqué

- Datasheet
 - Contient l'ensemble des informations matérielles
 - C'est votre seule et unique référence absolue



Attention à la différence entre la datasheet de la carte et la datasheet du microcontrôleur

Les documentations importantes à savoir reconnaître et utiliser efficacement :

- RM → Reference Manual, doc exhaustive sur un circuit en particulier (1000+ pages)
- UM → User Manual, doc plus simple et plus rapide à prendre en main sur un circuit ou une librairie (50 à 200 pages pour un circuit, 1000+ pages pour une librairie)
- DB → Data Brief, présentation succincte d'un circuit ou d'une carte (~10 pages)
- TN → Technical Note, présentation détaillée d'un point très particulier (~10 pages)
- AN → Application Note, présentation détaillée d'un exemple d'utilisation, souvent avec des codes exemples (10 à 100 pages)

GPIO : General Purpose Input Output

Définition : Les GPIOs (*General Purpose Input Output*) sont l'ensemble des pins programmables du microcontrôleur. Ce sont des couteaux suisses car chaque pin peut avoir plusieurs fonctions (mais pas simultanément). Globalement, tous les pins du microcontrôleur sont des GPIOs, à l'exception des pins d'alimentation et de *reset*. On distingue plusieurs modes :

En entrée haute-impédance (*input high-Z*) :

- Numérique
- Analogique

En sortie (*output*) :

- Push-Pull (PP)
- Open-Drain (OD)
- PWM (*Pulse Width Modulation*)
- Analogique

De plus, sur certains modes, il peut y avoir une résistance de tirage (*Pull-up* ou *pull-down*). Ceci est un réglage indépendant du mode.

GPIO : Vitesse de balayage (*slew rate*) & consommation

On représente les fronts comme verticaux et donc instantanés, mais en réalité, les fronts sont étalés sur un temps dépendant de la charge du pin. C'est le cas dans tous les modes de sortie, et encore plus dans le cas Open Drain car la résistance de tirage est en général de l'ordre du $k\Omega$. La vitesse de balayage (*slew rate*) s'exprime en $MV s^{-1}$.

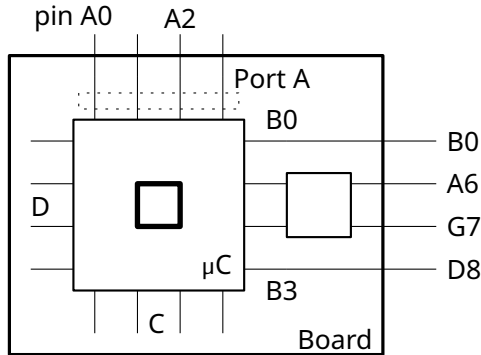
Chaque pin a une consommation maximale, de l'ordre de la dizaine de mA. Le microcontrôleur a une consommation maximale de l'ordre de la centaine de mA.

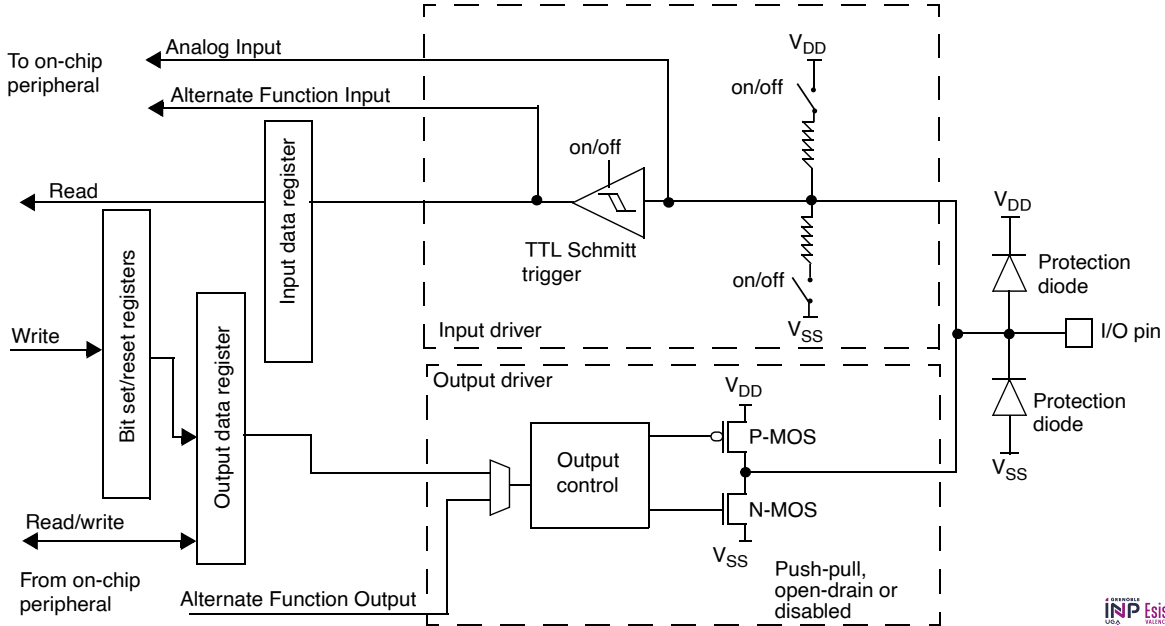
 Il faut donc veiller à ce que la somme des consommations de chaque GPIO ne dépasse pas la consommation maximale du microcontrôleur.

GPIO : Port vs pin

Les GPIOs sont regroupés par port et chaque port contient plusieurs pattes (*pins*), général 8 ou 16. Les ports sont désignés par des lettres (e.g. A à F) et les pattes par des nombres (e.g. 0 à 15). Ainsi, le pin PB5 référence le port B et le pin 5.

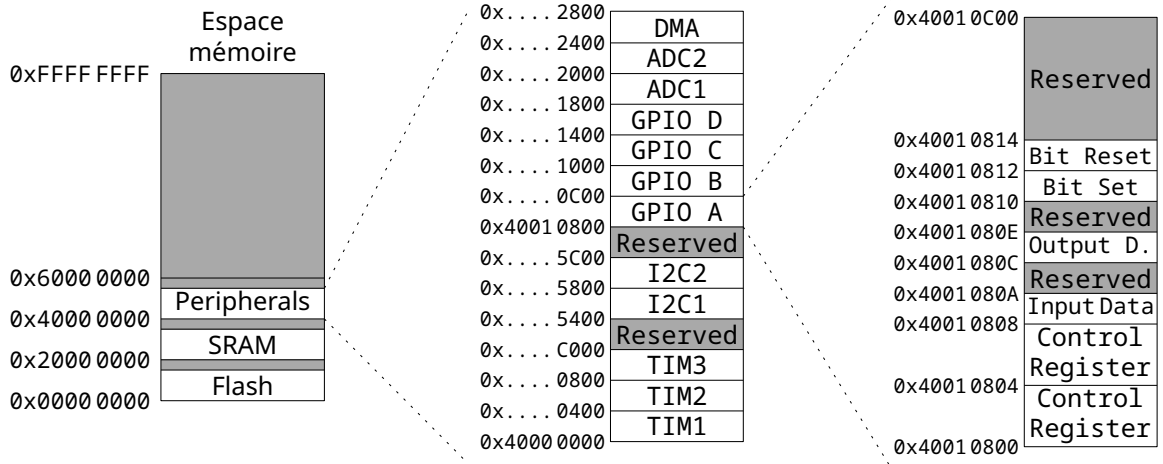
Les pins sont indépendants les uns des autres à l'exception des interruptions. Chaque port a ses propres registres de configuration qui regroupent l'ensemble des pins du port.





Il y a 2 possibilités pour adresser les périphériques : soit par la mémoire (*memory-mapped IO*), soit par des instructions spécifiques (*port-mapped IO*) normalement bien nommées *in* et *out*.

Programmation des périphériques : Adressage mémoire (*memory-mapped IO*)



9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G) Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

CNFy[1:0]: Port x configuration bits (y= 0 .. 7)

These bits are written by software to configure the corresponding I/O port.

Refer to [Table 20: Port bit configuration table](#).

In input mode (MODE[1:0]=00):

00: Analog mode

01: Floating input (reset state)

10: Input with pull-up / pull-down

11: Reserved

In output mode (MODE[1:0] ≥ 00):

00: General purpose output push-pull

01: General purpose output Open-drain

10: Alternate function output Push-pull

11: Alternate function output Open-drain

MODEy[1:0]: Port x mode bits (y= 0 .. 7)

These bits are written by software to configure the corresponding I/O port.

Refer to [Table 20: Port bit configuration table](#).

00: Input mode (reset state)

01: Output mode, max speed 10 MHz.

10: Output mode, max speed 2 MHz.

11: Output mode, max speed 50 MHz.

Programmation des périphériques : Adressage par instructions spécifiques (*Port-mapped IO*)

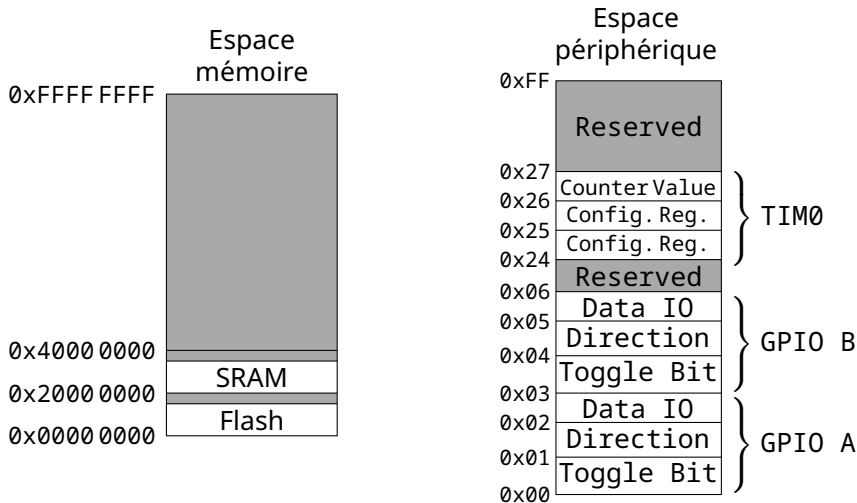


Table 13-1. Port Pin Configurations

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output low (sink)
1	1	X	Output	No	Output high (source)

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Adressage par la mémoire (*memory-mapped IO*) :

- + Plus simple à gérer (un seul espace mémoire)
- + Moins d'instructions matérielles → moins de silicium
- + Possibilité d'avoir de très nombreux périphériques
- Risques en cas de mauvaises manipulations de pointeurs

Adressage par instructions spécifiques (*port-mapped IO*) :

- + Instructions plus rapides → 1 cycle au lieu de 2 (Atmel)
- + Aucun risque sur les pointeurs
- Plus d'instructions matérielles → plus de silicium
- Nombre de périphérique limité

Programmation des périphériques : Drapeaux (*flags*)

Chaque bit ou groupe de bits d'un registre périphérique contrôle une fonctionnalité. Dans le cas du GPIO STM32, chaque groupe de 4 bits contrôle un pin, sous découpé en 2 groupes de 2 bits, dont le premier gère le mode (entrée ou sortie avec $\neq$ fréquences), et le second la configuration du mode (PP, OD en sortie ; pull-up, analog, flottant en entrée).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

CNFy[1:0]: Port x configuration bits (y= 0 .. 7)

In input mode (MODE[1:0]=00):

00: Analog mode

01: Floating input (reset state)

10: Input with pull-up / pull-down

11: Reserved

In output mode (MODE[1:0] > 00):

00: General purpose output push-pull

01: General purpose output Open-drain

10: Alternate function output Push-pull

11: Alternate function output Open-drain

MODEy[1:0]: Port x mode bits (y= 0 .. 7)

00: Input mode (reset state)

01: Output mode, max speed 10 MHz.

10: Output mode, max speed 2 MHz.

11: Output mode, max speed 50 MHz.

Comment changer un seul groupe de bits sans modifier les autres ?

Programmation des périphériques : Synoptique de code

```
// solution 1
// quelque part dans un header fourni
volatile uint32_t* GPIOA_CRL =
    ↪ GPIOA_BASE + 0x0; // pin 0-7
volatile uint32_t* GPIOA_CRH =
    ↪ GPIOA_BASE + 0x4; // pin 8-15
volatile uint32_t* GPIOA_IDR =
    ↪ GPIOA_BASE + 0x8; // input
volatile uint32_t* GPIOA_ODR =
    ↪ GPIOA_BASE + 0xC; // output

uint32_t tmp = *GPIOA_IDR; // lecture
*GPIOA_ODR = tmp;          // écriture
```

```
// solution 2
// quelque part dans un header fourni
typedef struct {
    volatile uint32_t CRL; // pin 0-7
    volatile uint32_t CRH; // pin 8-15
    volatile uint32_t IDR; // input
    volatile uint32_t ODR; // output
} GPIO_TypeDef;
#define GPIOA ((GPIO_TypeDef
    ↪ *)GPIOA_BASE)

uint32_t tmp = GPIOA->IDR; // lecture
GPIOA->ODR = tmp;          // écriture
```

Programmation des périphériques : Synoptique de code par scrutation (*polling*)

```
// solution 1
while(*GPIOA_IDR & 0x80 != 0x80) {
    f1(); f2();
    f3(); f4();
}
handle_GPIOA_event();
```

```
// solution 2
if(*GPIOA_IDR & 0x80 != 0x80) f1();
else handle_GPIOA_event();
if(*GPIOA_IDR & 0x80 != 0x80) f2();
else handle_GPIOA_event();
if(*GPIOA_IDR & 0x80 != 0x80) f3();
else handle_GPIOA_event();
if(*GPIOA_IDR & 0x80 != 0x80) f4();
else handle_GPIOA_event();
```

⚠ Si `fn()` est trop long, on rate des évènements. Si d'autres évènements (i.e. d'autres conditions) sont à surveiller, ça devient vite lourd, incompréhensible et *spaghetti*.

Comment éviter de passer son temps à regarder l'état d'une broche ? **Avec une interruption**

- Sensibles aux évènements, i.e. aux changements d'état $H \rightarrow L$ et/ou $L \rightarrow H$
- « Réveille » le CPU s'il dort ou interrompt la tâche en cours et lance une routine qui traite l'interruption
- Met un bit à 1 (*flag*) dans le registre d'interruption correspondant pour signaler la présence d'une interruption



à bien autoriser les interruptions (souvent plusieurs registres de configuration, un pour les interruptions globales, et un pour chaque type d'interruption)

- Une interruption par port (au mieux), si plusieurs pins sont configurés en interruption sur le même port, il faut lire l'état des pins pour déterminer lequel à déclencher l'interruption

- N'entre dans une interruption que si la priorité est inférieure à l'interruption en cours
 - Priorité réglable sur certains microcontrôleur (ceux basés sur Arm notamment)
- Saute à une adresse fixe dans le code, la table d'interruption (Interrupt Vector Table)
- Sauvegarde les registres (y compris SR), parfois automatiquement
- Penser à les restaurer à la fin de l'interruption (parfois auto)
- Parfois nécessaire d'écrire le bit d'interruption pour dire qu'on l'a traitée (et autoriser de futures interruptions)

Une interruption :

- ne prend pas de paramètre et n'a pas de valeur de retour
- utilise des variables `volatile` et globales
- met uniquement à jour ce qui doit l'être
- doit être la plus courte possible

Interruptions : Synoptique de code par interruption

// solution 1

```
void HAL_GPIO_IRQ() {  
    gpio_event = 1;  
}
```

// dans la boucle principale

```
if(gpio_event) {  
    gpio_event = 0;  
    handle_event();  
}
```

// solution 2

```
void HAL_GPIO_IRQ() {  
    handle_event();  
}
```

// dans la boucle principale

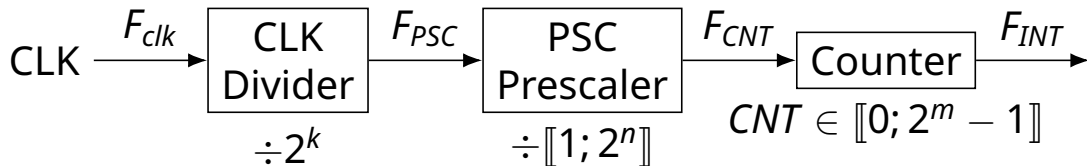
```
/** rien **/
```

 Si `handle_event` est trop long, on peut toujours rater des évènements.

Les *timers* sont le second type de périphérique le plus commun. On en trouve dans tous les microcontrôleurs. Ils servent à mesurer le temps et peuvent fonctionner en plusieurs modes :

- Base de temps
- Input capture
- Output Compare ou PWM

Timers : Schéma bloc interne



Les réglages en général disponibles sur les timers basiques sont :

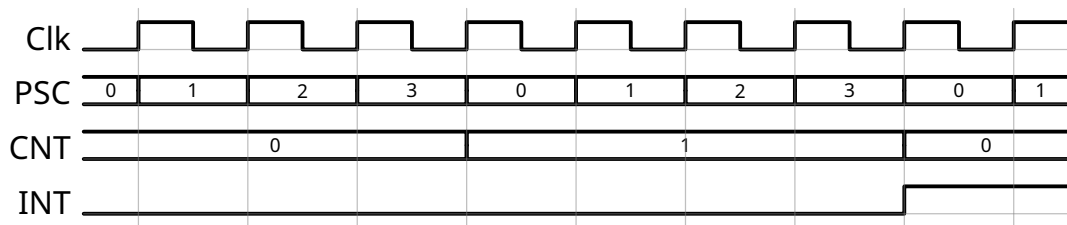
- la valeur de la division d'horloge, **si elle existe**, en général dans une liste prédéfinie et toujours une puissance de 2
- la valeur de division du prescaler entre 1 et N (N puissance de 2)

⚠ On règle le prescaler à la valeur N et il divise par N + 1 ⚠

⚠ Parfois la division d'horloge est appelée *prescaler*, dans ce cas, le +1 ne s'applique pas ⚠

- la valeur maximale de comptage entre 0 et $2^m - 1$ (mode compteur ou centré-aligné) OU la valeur de démarrage (en mode décompteur)

Timers : Chronogramme



Prescaler réglé sur 3 (donc divise la fréquence par 4), valeur max de comptage à 1

- L'interruption est générée au moment où le compteur repasse à 0, ou à valeur max si décomptage, ou les 2 en configuration centré-aligné
- La valeur interne de comptage du prescaler n'est jamais accessible, seule la valeur maximale est accessible et réglable
- La valeur de comptage du timer et sa valeur maximale sont toujours accessibles

Timers : Configuration basique

3 configurations possibles :

- Compteur (vers le haut)
- Décompteur (vers le bas)
- Centré aligné (vers le haut puis vers le bas)

Dans tous les cas, compte ou décompte de 1 en 1.

On a donc 3 (ou 2 si pas de division d'horloge) valeurs à choisir :

- Essayer d'avoir une division entière de F_{clk} sur une unité facile à manipuler comme 1 μs ou 100 ns
- Plusieurs configurations sont possibles entre le prescaler et la valeur maximale pour la même période
- En général, on essaye d'avoir des divisions entières pour éviter les erreurs d'arrondi

En fonctionnement compteur ou décompteur :

$$T_{timer} = \frac{1 + MAX}{F_{CNT}}$$

$$F_{CNT} = \frac{F_{clk}}{DIV_{clk} \times (PSC + 1)}$$

$$\Rightarrow MAX = \frac{T_{timer} \times F_{clk}}{DIV_{clk} \times (PSC + 1)} - 1$$

En fonctionnement centré-aligné :

$$T_{timer} = \frac{2 \times MAX}{F_{CNT}}$$

$$F_{CNT} = \frac{F_{clk}}{DIV_{clk} \times (PSC + 1)}$$

$$\Rightarrow MAX = \frac{T_{timer} \times F_{clk}}{2 \times DIV_{clk} \times (PSC + 1)}$$

Timers : Exemple 1

On souhaite générer une base de temps de 1 s sur un microcontrôleur fonctionnant à 64 MHz et disposant d'un timer 16 bits qui lui-même dispose d'une division d'horloge et d'un prescaler de 16 bits également.

- Diviser l'horloge par 64 pour obtenir une fréquence F_{PSC} de 1 MHz
- Mettre le prescaler à 999 pour obtenir une fréquence F_{CNT} de 1 kHz
- Mettre une valeur maximale de comptage à 999 pour obtenir une fréquence F_{INT} de 1 Hz

Permet d'obtenir une horloge système de granularité 1 ms qui compte le nombre de secondes depuis le démarrage du timer.

Timers : Exemple 2

On souhaite générer une base de temps de 1 ms sur un microcontrôleur fonctionnant à 100 MHz et disposant d'un timer 16 bits qui lui-même dispose d'une division d'horloge et d'un prescaler de 16 bits également.

- Il n'y a pas de division entière, donc on peut ne pas diviser l'horloge et avoir $F_{PSC} = F_{clk} = 100 \text{ MHz}$
- Mettre le prescaler à 999 pour obtenir une fréquence F_{CNT} de 100 kHz
- Mettre une valeur maximale de comptage à 99 pour obtenir une fréquence F_{INT} de 1 kHz
- Autres couples possibles : 9 et 9999, 4 et 19999, etc. Tout couple d'entiers vérifiant $(PSC + 1) \times (MAX + 1) = \frac{F_{INT}}{F_{clk}} = 100\,000$ sous la contrainte des 16 bits

Timers : Configuration

- Identifier les registres et la configuration souhaitée ($\nearrow$, $\searrow$, $\nearrow\searrow$)
- Arrêter le timer s'il tourne déjà
 - Évite les interruptions inutiles et de potentiels verrouillages (*deadlocks*)
- Configurer les registres
- Nettoyer (*clear*) le bit d'interruption
- Faire une remise à zéro du timer ET du prescaler
 - Parfois fait automatiquement lors de l'écriture de certains registres
- Lancer le timer
 -  Toujours à faire absolument en dernier

Timers : Interruption

En général, l'interruption d'un timer ne fait qu'incrémenter un compteur :

```
volatile uint32_t timer = 0;
void TIM2_ISR(void) {
    timer++;
}
// in the main infinite loop
// active waiting (not the best)
while(timer - lasttimer < period);
doSomethingPeriodic();
```

 Attention au dépassement de capacité de votre variable qui gère le temps !

Type \ F _{INT}	F _{INT}		
	1 μ s	1 ms	1 s
uint8_t	256 μ s	256 ms	4m16s
uint16_t	65 ms	1m5s	18h20m
uint32_t	1h11m	~ 50 jours	136 ans
uint64_t	585 ka	585 Ma	585 Ga

Timers : Choix de l'horloge

Il est possible de choisir parmi plusieurs horloges pour un timer :

- Horloge interne
- Horloge externe sur pin GPIO (*external clock source*)
 -  Les horloges externes ne doivent pas être plus rapides que l'horloge interne
 -  Les horloges externes n'ont pas toujours accès à un prescaler ou une division d'horloge
 - Choix de la polarité possible
- Déclenchement externe sur GPIO (*external trigger input*)
 - Attend un front externe puis commence à compter
- Déclenchement interne (*internal trigger input*)
 - Utilise un timer comme prescaler pour un autre timer

Timers : Input Capture

- Capture (i.e. enregistre) la valeur du compteur dans un registre sur un front d'un signal externe
 - Filtre possible sur le signal externe (évite les rebonds)
- Déclenche une interruption
 - Il n'y a qu'un registre de capture, il faut donc le sauvegarder pour ne pas perdre la donnée lors de la prochaine capture
- Remise à 0 du timer (optionnel)

 Le logiciel (donc **VOUS**) doit gérer le cas où le timer déborde entre 2 captures

 Il faut évidemment régler le timer comme vu précédemment (prescaler, division d'horloge, valeur maximale)

Timers : Input Capture : exemple d'application avec un mètre ultrasonique



igger



Timers : Output Compare

- Compare la valeur du compteur à une ou plusieurs valeurs
 - Contrôle un pin en fonction du résultat de comparaison
 - 1 si $cpt < val$ et 0 sinon (et inversement) $\rightarrow$ MLI (*PWM*)
 - 1 si $cpt = val$, 0 sinon (et inversement) $\rightarrow$ génération de pulses
 - Inverse le pin si $cpt = val \rightarrow$ MLI (*PWM*)
 - Interruption possible quand $cpt = val$ et au dépassement (*overflow*) également
-  Il faut évidemment régler le timer comme vu précédemment (prescaler, division d'horloge, valeur maximale)

Timers : Output Compare

- Configuration compteur

- Rapport cyclique (*duty cycle*) $\alpha = \frac{OCR}{1+MAX}$, OCR est la valeur de comparaison

- Période PWM : $T_{PWM} = T_{Timer} = \frac{(1+MAX) \times DIV_{clk} \times (PSC+1)}{F_{clk}}$

- Configuration centré-aligné

- Rapport cyclique (*duty cycle*) $\alpha = \frac{OCR}{MAX}$

- Période PWM : $T_{PWM} = \frac{2 \times MAX \times DIV_{clk} \times (PSC+1)}{F_{clk}}$

- On peut jouer sur l'alignement et l'asymétrie entre 2 PWMs
 - 2 PWMs avec même timer centré-aligné
 - Utilise le 1^{er} canal en montant et le 2nd canal en descendant
- Possibilité de combiner les PWM entre eux (ET/OU logique)
 - 2 PWMs avec même timer
 - Permet de contrôler la phase du signal résultant
- Possibilité d'ajouter des temps morts
 - Pilotage de composants de puissance triphasés
 - Évite les court-circuits transitoires (~ 10 ns)

Timers : Conclusion

3 modes de fonctionnement

- Base de temps
 - Réglage période/fréquence avec division d'horloge, prescaler et valeur maximale
 - Réglage mode de comptage: ↗, ↘ ou ↗↘ (centré-aligné)
 - Input Capture
 - Configurer pin d'entrée
 - Régler la polarité (front ↑, ↓ ou les 2)
 - Output Compare
 - Configurer pin de sortie
 - Régler le mode et la valeur de comparaison
- } ⇒ Utilise la base de temps

Ne pas oublier d'autoriser les interruptions si besoin est et bien évidemment de lancer le timer en dernier

- La fréquence n'est **jamais** exactement X MHz
- La fréquence n'est **pas** fixe
 - Variations dépendantes de la stabilité de la tension d'alimentation
 - Variations liées à la température
 - Variations dues au vieillissement
- Fréquence du cristal ou de l'oscillateur exprimée avec une tolérance (en ppm ou en %)



Erreur de mesure intrinsèque de $\frac{T_{CNT}}{2}$

CAN : Convertisseur Analogique Numérique (*Analog Digital Converter, ADC*)

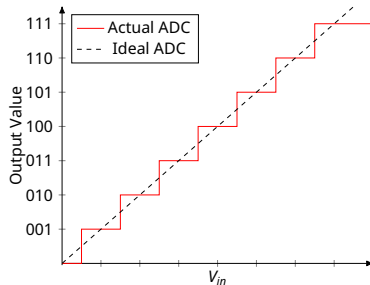
- Circuit permettant de convertir une tension analogique en un nombre

⚠ La tension à convertir doit se trouver dans les bornes d'alimentation du CAN, en général entre 0 V et V_{DD} (3.3 V ou 5 V)

- Le résultat est un nombre entier non signé compris entre 0 et $2^n - 1$ pour un convertisseur n bits

Les paramètres d'importance sont :

- La fréquence d'échantillonnage F_{ech}
- La résolution n (nombre de bits)
- La linéarité (en tension et en fréquence)
- Le type de convertisseur



CAN : Convertisseur Analogique Numérique (*Analog Digital Converter, ADC*)

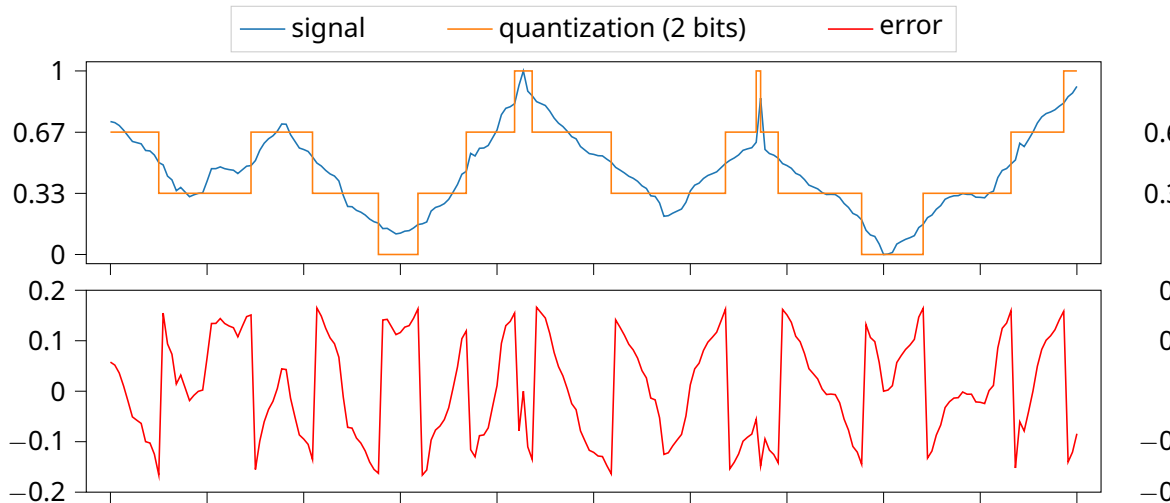
Le résultat d'une conversion est définie par :

$$N = (2^n - 1) \times \frac{V_{in} - V_{min}}{V_{MAX} - V_{min}} \simeq 2^n \times \frac{V_{in} - V_{min}}{V_{MAX} - V_{min}}$$

On définit le pas de quantification (*quantum*) $q = \frac{V_{MAX} - V_{min}}{2^n - 1} \simeq \frac{V_{MAX} - V_{min}}{2^n}$

Par construction, tout CAN a une erreur intrinsèque (erreur de quantification) de $e = \frac{q}{2}$

CAN : Effet de la quantification



CAN : Types de convertisseur

- Convertisseur simple ou double rampe
 - Basé sur la charge/décharge d'une capacité à courant constant
 - Simple, peu cher, relativement lent
- Convertisseur Delta-Sigma
 - Suréchantillonnage puis décimation
 - Très précis, $\sim 1 \text{ MS/s}$, application audio
- Convertisseur à approximations successives (*SAR*)
 - Basé sur la dichotomie
 - $< 10 \text{ MS/s}$, $\sim 12 \text{ bits}$
 - Majorité des CAN sur microcontrôleur
- Convertisseur Flash
 - Réseau de résistance R qui crée toutes les tensions intermédiaires
 - Cher (forte contrainte sur la précision des résistances), très rapide, peu précis (8 bits)
- Chaque convertisseur a en entrée une capacité commutée et un filtre passe-bas

CAN : Ou en trouve t-on ?

- Micros et caméras
- Capteurs analogiques (sonde de température PT-100, CTN 10k, etc.)
- Capteur de précision (jauge de contrainte, etc.)
- Écran tactile
- Radiocommunication (4G, 5G, Wifi, Bluetooth, TNT, etc.)

CAN : Fonctionnalités avancées

- Interruption à la fin d'une conversion
- Chien de garde analogique (*analog watchdog*)
 - Déclenche une interruption si la tension sort des bornes fixées
- Conversion multiple (*scan mode*)
 - pin 1, puis pin 2, puis pin 3, puis pin 1, etc.
- Conversion continue
- Conversion différentielle
- Conversion référencée
 - Permet d'augmenter la précision de la conversion
- Conversion double (*dual mode*)
 - pin 1 ADC1, pin 1 ADC2, pin 1 ADC1, etc.
 - Augmente la fréquence de conversion
- Déclenchement externe

CAN : Précautions d'usages

- Alignement droite ou gauche des résultats de conversion et à la précision
 - 10 ou 12 bits de résultat dans un registre de 16 bits
- Vérifier que le CAN a fini sa conversion en cours avant d'en lancer une nouvelle (sauf dans l'interruption)
- Respecter le délai de conversion avant de lire le résultat
- Calibration parfois nécessaire après chaque démarrage du microcontrôleur
- Certains timers sont parfois couplés aux CANs

La grandeur que l'on souhaite mesurer est rarement la tension elle-même, on cherche plutôt une grandeur physique qui influence directement l'impédance d'un circuit (et donc la tension ou la fréquence). On doit donc utiliser des circuits pour conditionner et amplifier le signal à mesurer :

- Pont diviseur de tension
- Pont de Wheatstone (quart, demi, entier)
- Amplificateur inverseur/non-inverseur
- Amplificateur différentiel
- Amplificateur d'instrumentation

CAN : Bruit & erreur

Une chaîne d'acquisition complète comporte un capteur, un conditionnement, une amplification du signal et une acquisition. Du bruit peut s'ajouter dans chacun de ces éléments qui, eux-mêmes ne sont pas parfaits et ont donc une erreur.

Les principales sources de bruits sont :

- Bruit du capteur
- Bruit thermique (de Johnson, bruit blanc)
- Bruit de grenaille (de Schottky, bruit rose, *shot noise*)
- Couplage et CEM
- Bruit de quantification
- Bruit de scintillation (*flicker*), bruit poivre et sel ou impulsif, bruit en créneaux (*burst noise*), bruit d'avalanche

Les principales sources d'erreurs sont :

- Erreur de modélisation
- Erreur de valeur des composants
- Dérive temporelle

CAN : Propagation des incertitudes

Dans le cas général, si on a $y = f(x_1, x_2, \dots, x_n)$ avec x_i tous indépendants, alors :

$$\Delta y^2 = \sum_{i=1}^n \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2$$

On peut aussi calculer le pire cas :

$$\Delta y = \sum_{i=1}^n \left| \frac{\partial f}{\partial x_i} \right| \Delta x_i$$



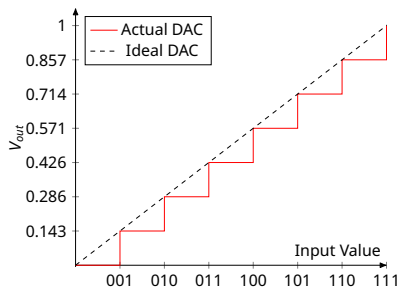
Ceci est valable pour toute mesure de quoi que ce soit, y compris les timers !

CNA : Convertisseur Numérique Analogique (*Digital Analog Converter, DAC*)

- Circuit permettant de convertir un nombre en une tension analogique
- Le nombre à convertir est un nombre entier compris entre 0 et $2^n - 1$
- La sortie est une tension comprise entre les bornes d'alimentation du CNA, en général entre 0 V et V_{DD} (3.3 V ou 5 V)

Les paramètres d'importance sont :

- La fréquence de fonctionnement
- La résolution n (nombre de bits)
- La linéarité (en tension et en fréquence)
- Le type de convertisseur



CNA : Convertisseur Numérique Analogique (*Digital Analog Converter, DAC*)

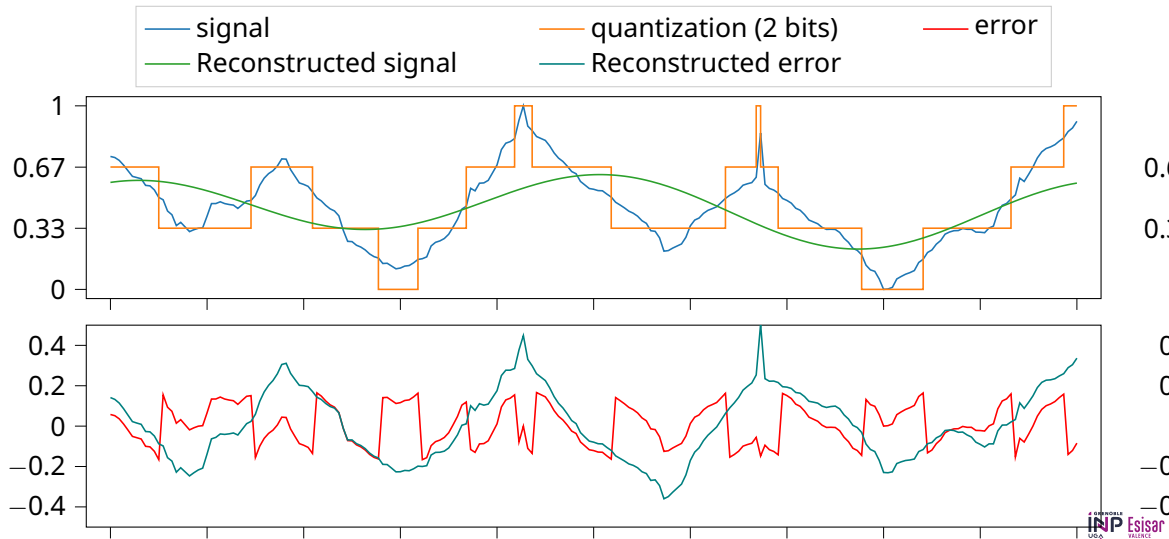
Le résultat d'une conversion est définie par :

$$V_{\text{out}} = \frac{N}{2^n - 1} \times (V_{\text{MAX}} - V_{\text{min}}) + V_{\text{min}} \simeq \frac{N}{2^n} \times (V_{\text{MAX}} - V_{\text{min}}) + V_{\text{min}}$$

On a la même définition du pas de quantification (*quantum*) que pour le CAN :

$$q = \frac{V_{\text{MAX}} - V_{\text{min}}}{2^n - 1} \simeq \frac{V_{\text{MAX}} - V_{\text{min}}}{2^n}$$

CNA : Effet de la quantification



La plupart des méthodes CAN fonctionnent aussi avec les CNA :

- Convertisseur simple ou double rampe
- Convertisseur Delta-Sigma
- Convertisseur Flash (R/2R)

À cela, on peut rajouter le pseudo analogique avec le PWM.

CNA : Où en trouve t-on ?

- CAN
- Haut-parleurs
- Générateur de signaux
- Radiocommunication

- Interruption à la fin d'une conversion
- Générateur de bruit (LFSR)
- Générateur triangulaire
- Déclenchement externe

- Alignement droite ou gauche de la conversion et précision
 - 8, 10 ou 12 bits dans un registre de 16 bits
- Vérifier que le CNA a fini sa conversion en cours avant d'en lancer une nouvelle (sauf dans l'interruption)
- Respecter le délai de conversion
- Prévoir un étage d'adaptation avec un montage suiveur pour ne pas tirer trop de courant
- Prévoir également une adaptation d'impédance selon l'application
- Prévoir un filtre passe-bas pour lisser la sortie du CNA si besoin est

DMA : Direct Memory Access

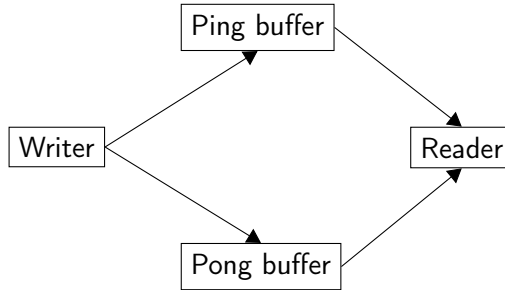
Comment éviter avec le CAN ou le CNA pleins de petites interruptions? **Avec un DMA**

- Périphérique permettant de déplacer automatiquement des données entre les périphériques et la mémoire sans interrompre le CPU
- Plusieurs canaux disponibles
 - Un canal représente un transfert entre une source et une destination
- Priorité réglable pour chaque canal (4 niveaux sur STM32)
 - Si même priorité en cas de conflit, priorité au canal ayant l'ID le plus faible
 - En cas de conflit d'accès entre le CPU et le DMA, le bus est partagé chacun son tour (*round-robin*)
- Les périphériques lancent le DMA automatiquement
- Chaque périphérique est matériellement relié à certains canaux
- 2 types de DMA : *fly-by* et *flow-through*

DMA : Configuration

- Taille des éléments à transférer : 1, 2 ou 4 octets
- Nombre d'éléments à transférer
- Adresse de départ et incrément
- Adresse d'arrivée et incrément
- Mode circulaire ou non
- Changement d'endianness et de taille d'élément

DMA : Ping-pong buffer



- Après chaque transfert, on alterne les buffers
 - Interruption de fin de transfert qui met à jour les pointeurs
- À vous de garantir que le temps de traitement est inférieur au temps entre 2 transferts

L'horloge est un périphérique comme les autres. Il est possible de sélectionner différentes sources d'horloge :

- Oscillateur interne (LC) rapide ou lent
- Oscillateur externe (cristal) rapide ou lent
- PLL avec choix du multiplieur

La consommation du microcontrôleur est en général proportionnelle à la fréquence. La datasheet peut fournir une consommation en $\mu\text{A}/\text{MHz}$ ou $\mu\text{W}/\text{MHz}$.

Il y a également plusieurs domaines d'horloge dans un microcontrôleur :

- Horloge CPU
- Horloge périphériques
- Horloge ADC

- Le contrôle de puissance est un périphérique comme un autre
- Il y a plusieurs niveaux de puissance (4 sur STM32)
 - Run
 - Sleep : CPU éteint, tous les périphériques restent allumés
 - Stop : CPU éteint, toutes les horloges éteintes
 - Standby : tout éteint, y compris mémoire et registres CPU

 Parfois les mémoires sont également éteintes → Données perdues

 Bien regarder si les évènements qui nous intéressent réveillent le μC

 Délai de réveil long

- RTC (*Real Time Clock*)
- Chien de garde (*Watchdog*)
- Brown-out et black-out
- Tous les périphériques spécifiques à certaines applications
 - Codec Audio/Vidéo
 - Accélérateurs matériels (DSP, FFT, Cryptographie, traitement d'images, etc.)

Autres périphériques particuliers : RTC

- Timer particulier qui conserve sa valeur même après un reset ou une perte d'alimentation
- Permet de conserver l'heure et la date
- Continue à fonctionner même lorsque le système est éteint (sous réserve d'alimentation par une pile par exemple)



© Wikimedia-Commons, Afrank99,
Coin-cells.jpg, © i i

Autres périphériques particuliers : Chien de garde (*Watchdog*)

- Timer particulier qui déclenche un reset complet lorsqu'il atteint 0
- En fonctionnement normal, on remet le WDT (WatchDog Timer) à une valeur N régulièrement
- En cas d'erreur dans le code ou de faute matérielle, le WDT n'est jamais remis à N et atteint 0 → le système redémarre

Autres périphériques particuliers : Brown-out et black-out

- Interruptions particulières qui indiquent au MCU que la tension d'alimentation devient trop basse pour assurer un fonctionnement normal
- Le constructeur vous garantit un certain temps et nombre d'opérations (notamment d'accès mémoire)

- Les périphériques à connaître et à savoir manipuler/configurer dans les bons modes selon l'application :
 - GPIO
 - Timers
 - ADC/DAC
 - DMA
- Les interruptions
- Bruits & erreurs
 - Bruit thermique
 - Bruit de quantification
 - Propagation des incertitudes

Plan

① Les microcontrôleurs

② Spécificités logicielles

③ Les périphériques

④ Bus et liaisons

Rappel du modèle OSI

UART

I2C

SPI

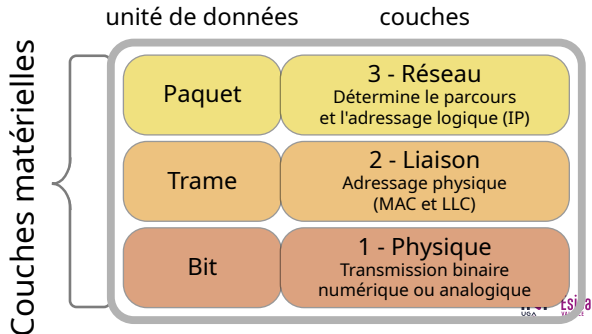
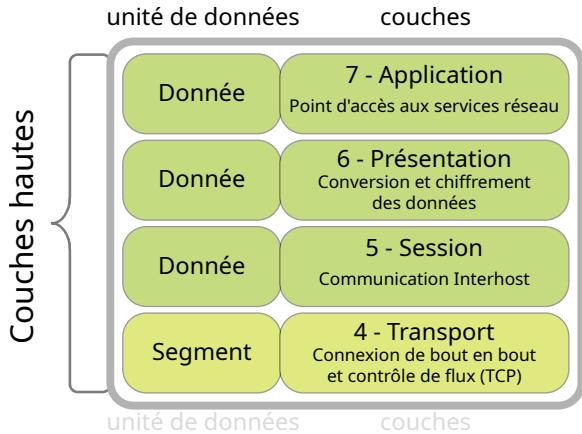
CAN

USB

Ethernet

⑤ Génie logiciel embarqué

Rappel du modèle OSI



Rappel du modèle OSI : Couche 3 Réseau

- Routage et adressage entre l'émetteur et le destinataire
 - Ajout d'une adresse pour indiquer le destinataire et/ou l'émetteur
- Fait en sorte que le message « Salut Alice, ça va ? » soit bien reçu par Alice et non par Bernard

Rappel du modèle OSI : Couche 2 Liaison de données

- Objectifs de détection ou de correction des erreurs
 - Ajout d'information en lien avec le protocole (bit de parité, bit de start, etc.)
 - Définit l'ordre de l'information, le temps du symbole, etc.
 - Définit le niveau de repos du médium
 - Gestion de l'accès au médium physique en cas de conflit
- Normalisation du « langage »
- Fait en sorte que le message « salut, ça va ? » soit bien compris comme « salut, ça va ? » et non comme « ? av aç ,tulas »

Rappel du modèle OSI : Couche 1 Physique

- Ne traite que des informations simples (typiquement 1 bit)
 - Niveaux de tension et temps du symbole
 - Encodage du symbole (Manchester, NRZ, etc.)
 - Medium de transmission (air, câble coaxial, simple fil, etc.)
 - Connecteurs
- Normalisation des interfaces
- Fait en sorte que le bit '0' soit lu comme '0' et non comme '1'

Définition : Liaison physique

Architecture de communication qui englobe la partie matérielle (cf couche 1 du modèle OSI)

- Plusieurs topologies possibles
 - Point à point
 - Bus
 - Étoile
- Plusieurs media possibles :
 - Monofilaire
 - Bifilaire en mode différentiel
- En plus du media, définit :
 - Le mode d'accès (PP ou OD)
 - Les terminaisons de ligne
 - Les répéteurs
 - Les isolations optiques ou galvaniques

Définition : Protocole

Ensemble des règles qui permet d'assurer la transmission correcte des données (cf couche 2 et 3 du modèle OSI)

- Signal de début de trame
 - Bit ou condition de start
 - Mot magique
- En-tête (*header*) de trame
 - Adresse destinataire
 - Taille
 - Type de requête
- Gestion des conflits (en lien avec couche 1)
- Queue (*footer*) de trame
 - Checksum
- Fin de trame
 - Bit ou condition de stop
 - Niveau de repos du medium

- Liaison point à point
 - UART/USART
 - USB
- Liaison bus
 - SPI
 - I2C
 - CAN
 - Ethernet
- Bus industriels
 - Modbus
 - Profibus
 - ARINC429
- Tous ces bus sont séries !
 - Câble sur plusieurs mètres voire km

Définition : Canal de communication

Ensemble constitué d'un protocole et d'une liaison physique pour transférer de l'information. Ses caractéristiques principales sont :

- Débit
 - Baudrate → nombre de symbole par seconde
 - Bitrate → nombre de bits par seconde
- Distance
- Latence
- Duplex
 - Simplex → communication unidirectionnelle
 - Half-Duplex → communication bidirectionnelle mais pas simultanément
 - Full-Duplex → communication bidirectionnelle simultanée

Différents encodages

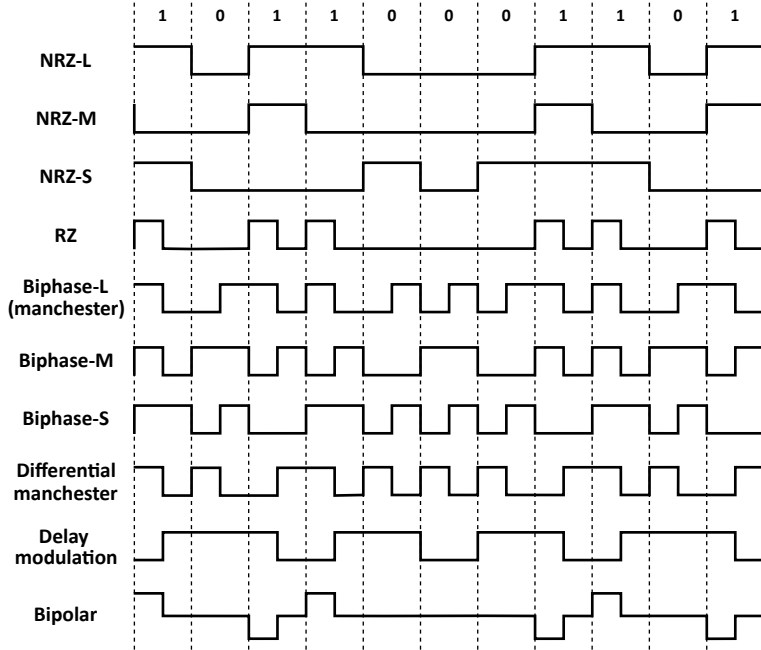
L'information peut être représentée physiquement par plusieurs grandeurs d'un même signal :

- Intensité
- Fréquence
- Phase

Dans le cas des communications filaires embarquées, on n'utilise que les variations d'intensité du signal électrique pour transmettre l'information.

Les différents codages possibles sont :

- NRZ (*Non Return to Zero*)
 - NRZI (*Non Return to Zero Inverted*)
 - RZ (*Return to Zero*)
 - Manchester
 - 8b/10b
- } Unipolaire ou Bipolaire
Absolu ou Différentiel



UART : Universal Asynchronous Receiver Transmitter

- 2 fils de communication (TX/RX) et un fil de masse. En pratique 1 fil Vcc également pour alimenter le composant
- Codage NRZ avec niveau de repos à 1
- Transmission Full-Duplex
- Liaison point à point
- Asynchrone

UART : Universal Asynchronous Receiver Transmitter

- Une trame contient :
 - Bit de start à 0
 - 5 à 9 bits de données (du lsb au msb)
 - Éventuellement un bit de parité (paire ou impaire)
 - 1, 1.5 ou 2 bits de stop à 1
- On définit l'efficacité du protocole $\eta = \frac{N_{\text{data}}}{N_{\text{transmis}}}$

UART : Universal Asynchronous Receiver Transmitter

- Pas d'horloge → Asynchrone
- Fréquence de fonctionnement en baud parmi une liste prédéfinie
 - 9600 baud, 19200 baud, ..., 115200 baud, ...
 - En pratique $\pm 10\%$ de tolérance



Le récepteur et l'émetteur doivent utiliser la même configuration

- Même taille de données (typiquement 8 bits)
- Même parité (en général, sans)
- Même taille de stop (typiquement 1 bit)
- Même baudrate

USART : Universal Synchronous and Asynchronous Receiver Transmitter

- Identique à l'UART
- 1 fil de plus pour transmettre ou recevoir l'horloge
- Avantage
 - Vitesse de transmission potentiellement variable
- Inconvénient
 - Un fil de plus
 - Consommation plus grande
 - Toujours besoin d'avoir la même configuration (sauf pour la fréquence)

UART : Lien vitesse de transmission et longueur de câble (RS-232)

Débit (baud)	Distance (m)
100	2000
2 400	60
4 800	30
9 600	15
19 200	7,6
38 400	3,7
56 000	2,6

- Utilisation d'un périphérique UART/USART si existant, sinon avec des GPIO et des timers (*bit-banging*)
- 3 modes possibles
 - Scrutation (*Polling*)
 - Interruption
 - DMA

UART : Utilisation en Polling lecture

- Avec périphérique
 - Lire le bit de réception en permanence
 - Lorsqu'il change d'état, lire le registre de donnée
 - Remettre le bit de réception à 0 (ou mis automatiquement par le matériel)
- Sans périphérique
 - Lire le GPIO en permanence et attendre le bit de start
 - Attendre la moitié du temps de symbole
 - Attendre le temps de chaque symbole, lire le GPIO et décaler le résultat

UART : Utilisation en polling écriture

- Avec périphérique
 - Lire le bit de transmission et attendre la fin d'une éventuelle transmission
 - Copier la donnée dans le registre de transmission
 - Sans périphérique
 - Envoyer le bit de start
 - Attendre le temps d'un symbole
 - Envoyer la donnée bit à bit
- } Répéter n fois
- Calculer la parité si nécessaire et l'envoyer
 - Envoyer le bit de stop

Avantages

- Fonctionne avec ou sans périphérique

Inconvénients

- Fonctions bloquantes
- Aucun moyen de savoir s'il y a eu transmission quand on ne regarde pas

UART : Interruption en lecture

- Avec périphérique
 - Lire la donnée depuis le registre de réception
 - Attendre la prochaine interruption
- Sans périphérique
 - 1ère interruption sur GPIO: bit de start, déclenche timer de 156 μ s (9600 bauds), arrêter l'interruption du GPIO
 - 2ème et + interruption du timer, lire le GPIO et décaler le résultat, mettre timer sur temps de symbole, 104 μ s
 - Remettre interruption sur GPIO et front descendant

UART : Interruption en écriture

- Avec périphérique
 - Copier la donnée dans le registre de transmission
 - Attendre l'interruption de fin de transmission
- Sans périphérique
 - Mettre bit de start et déclencher timer sur temps de symbole : $104\ \mu\text{s}$ (9600 bauds)
 - 2ème et + interruption du timer, mettre le bit lsb de la donnée et la décaler
 - 10ème interruption : mettre bit de stop et arrêter le timer

Avantages

- Non bloquant
- Impossible de rater une réception

Inconvénients

- Interruptions assez nombreuses, peut coûter cher en temps avec un haut baudrate
- Machine d'état nécessaire sans périphérique (code plus complexe)

- Uniquement avec périphérique
- Évite les interruptions pour chaque octet quand on en envoie plusieurs
- Une seule et unique interruption à la fin d'une transmission complète

UART : Exemple d'utilisation

- Communication entre 2 μ C (partage de tâches)
- Entre le μ C et le PC via le port USB
- Entre le μ C et le PC via bluetooth
- Il y a très longtemps
 - Entre le PC et la souris, le clavier ou l'imprimante



Avantages :

- Liaison extrêmement simple
- Largement répandu
- Détection d'erreur possible

Inconvénients :

- Peu rapide
- Non différentiel
- Pas de correction d'erreur
- Ne définit aucun protocole associé

I2C : Inter-Integrated Circuits / TWI (Two Wire Interface)

- 1 fil de donnée et 1 fil d'horloge avec résistances de tirage (+ masse)
 - SDA (*Seria DAta*) pour les données
 - SCL (*Serial CLock*) pour l'horloge
 - Masse (*ground*) pour référence de tension commune
- Relation maître-esclave avec 1 ou n maîtres et 1 ou n esclaves
- On transmet toujours du MSB au LSB
- Codage NRZ
- Transmission Half-Duplex
- Liaison bus
- Synchrone

- Condition de start mise par le maître
- Le maître met l'adresse (7 bits) et 1 bit $R/\overline{W}$
- Si l'adresse correspond à un esclave (ou à un maître), il répond par un Ack(nnowledge)
- Le maître met la requête/commande (8 bits), Ack de l'esclave
- Le maître met la donnée (8 bits), Ack de l'esclave OU l'esclave répond et le maître Ack
- Condition de stop mise par le maître



En cas de NAck, le maître met toujours une condition de stop pour les autres périphériques qui écoutent la ligne

- Au repos, SDA et SCL à 1 (Open-Drain avec pull-up)
 - Condition de démarrage (*Start condition*) : on tire SDA à 0 puis on tire SCL à 0
 - Pour chaque bit (du msb au lsb)
 - on met le bit sur SDA et on met SCL à 1
 - on met SCL à 0
 - L'esclave répond par un ACK en tirant SDA à 0 (et le maître relâche SDA)
 - Si lecture, l'esclave prend la main sur SDA
 - Condition de stop (*Stop condition*) : on met SCL à 1 puis SDA à 1
- } Répéter N fois

La gestion des conflits (ou collisions) se fait par CSMA/CD (*Carrier Sense Multiple Access with Collision Detection*)

- Le 0 est prioritaire sur le 1 par construction (Open-Drain, fonction ET câblé)
- Si 2 maîtres parlent en même temps, priorité à l'adresse la plus faible
- Le reste du temps, les maîtres écoutent la ligne pour savoir quand parler

- Adresse 0 est réservée pour faire une transmission (*broadcast*) à tous sur le bus
- 7 bits d'adresse $\rightarrow$ 128 composants (-1 pour l'adresse 0) = 127
- Débit entre 100kbit/s à 3Mbit/s
- Distance de ~ 1 m maximum
- *Bit-banging* possible

Avantages

- 2+1 fils partagés par 127 composants
- Protocole très simple et très répandu

Inconvénients

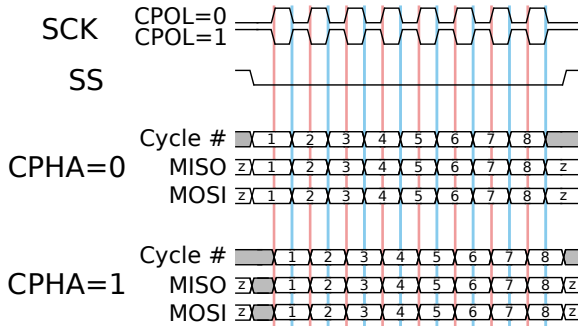
- Parfois certains composants ont la même adresse. . .
- Communication half-duplex

SPI : Serial Peripheral Interface

- 2 fils de communication, 1 fil d'horloge, 1 fil de sélection et la masse
 - MOSI (*Master Output Slave Input*) et MISO (*Master Input Slave Output*) pour communiquer
 - SCLK (*Serial CLock*) pour l'horloge
 - $\overline{SS}$ (*Slave Select*) pour sélectionner l'esclave
- Relation maître-esclave avec 1 maître et 1 ou plusieurs esclaves (n signaux SS)
- Codage NRZ
- Transmission Full-Duplex
- Liaison bus
- Synchrone

SPI : Serial Peripheral Interface

- Communication synchrone
- 4 configurations d'horloge
 - Polarité CPOL définit le niveau de repos de l'horloge
 - Phase CPHA définit si on lit sur 1^{er} ou 2nd front → front ↑ ou ↓ selon polarité
- En théorie, horloge avec rapport cyclique de 50%
- Débit jusqu'à 20Mbit/s



SPI : Exemple de trame complète

- Le maître sélectionne un esclave
 - Le maître génère l'horloge
 - Envoi d'une commande (8, 16 bits ou plus)
 - Envoi de l'adresse de la commande (8, 16 bits ou plus)
 - L'esclave répond avec la donnée correspondante
 - Le maître arrête l'horloge
 - Le maître relâche le signal $\overline{SS}$
- } Variable selon application

Avantages

- 3+1 fils partagés par autant de composants que nécessaire
- Protocole très simple et répandu
- Bon débit
- Communication Full-Duplex

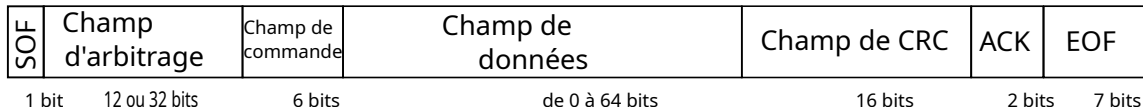
Inconvénients

- N fils nécessaires pour la sélection
- Courte distance uniquement (~ 30 cm)
- Pas d'acquittement

CAN : Controller Area Network

- 2 fils en paire différentielle CAN-L et CAN-H avec résistances de tirage et terminaison de ligne
- Tous les nœuds sont équivalents (pas de priorité)
- Protocole orienté message et non point à point
 - Tous les nœuds reçoivent le message et choisissent de le traiter ou de l'ignorer
- Codage NRZ (avec bourrage de bits)
- Transmission Half-Duplex
- Liaison bus
- Asynchrone
- 4 trames différentes : données, requête, erreur et surcharge

CAN : Trame de données



- Champ d'arbitrage détermine priorité du message
- Longueur des données contenu dans le champ de commande (en octet)
- CRC (*Cyclic Redundancy Check*) permet de détecter les erreurs
- 1er bit Ack est récessif pour l'émetteur mais tous les récepteurs émettent un bit dominant

- Au repos, CAN-L à H et CAN-H à L, états récessifs respectifs
- Bit de start : on tire CAN-L à L et on tire CAN-H à H
- On transmet ensuite chaque champ de la trame du MSB au LSB
- Les nœuds acquiescent à la fin en forçant 0, le transmetteur sait qu'au moins un nœud a reçu la trame
- Bits de stop : 7 bits récessifs et 3 bits d'inter trame récessifs
- Bourrage de bit (*bit stuffing*) si 5 bits identiques
 - Émission d'un bit de polarité inverse

Gestion des conflits par CSMA/CR (*Carrier Sense Multiple Access with Collision Resolution*)

- Le 0 est prioritaire sur le 1 par construction
- Si 2 nœuds parlent en même temps, priorité à l'ID de message le plus bas
- Le reste du temps, les nœuds écoutent la ligne

CAN : Informations générales

- 12 bits d'arbitrage → 4096 ID de messages
- Bon débit (> 1 Mbits/s)
- Distance de plusieurs dizaines de mètres grâce à la paire différentielle

Débit (kbit/s)	Distance (m)
1 000	30
800	50
500	100
250	250
125	500
62,5	1 000
20	2 500
10	5 000

Avantages

- 2 fils partagés par autant de composants que nécessaire



Charge du bus à commuter

- Protocole très répandu dans l'industrie (automobile, aéronautique, ascenseurs, etc.)
- Permet d'éliminer des nœuds défaillants via des compteurs d'erreur

Inconvénients

- Protocole assez complexe par rapport à l'I2C (trame de requête, d'erreur et de surcharge)
- Nécessité de bien choisir la priorité de chaque message en fonction de la criticité
- Protocole non déterministe
- Non sécurisé

USB : Universal Serial Bus

- 2 fils de communication (différentiel) avec *pull-down*, 2 fils d'alimentation
- Relation maître-esclave
- Codage NRZI (USB1, USB2)
- Codage 8b/10b (USB3)
- Transmission Half-Duplex
- Liaison point à point

- Fonctionnement en transaction découpée en paquets
 - Token (requête)
 - Paquet de donnée (en réponse ou en requête)
 - Acknowledge (confirmation)
- Chaque paquet contient :
 - Synchronisation sur 8 bits
 - PID sur 8 bits
 - Champs variable (jusqu'à 1024 octets)
 - Un CRC (taille variable)
 - End of Packet (EOP) sur 3 bits

- 2×2 paires différentielles (RX/TX) torsadées sur prise RJ45
- Codage Manchester (10Mbit/s)
- Codage 4b/5b ou 8b/10b (100Mbit/s)
- Transmission Full-Duplex
- Liaison point à point ou bus
- Plus de 100 normes ou formats associés

Ethernet : Trame

- Préambule sur 7 octets
- *Start Frame Delimiter* (SFD) sur 1 octet
- Adresse MAC destination (6 octets)
- Adresse MAC source (6 octets)
- EtherType (2 octets) : type de protocole correspondant à cette trame
 - IPV4 : 0x0800
 - IPV6 : 0x86DD
 - Profinet : 0x8892
 - EtherCAT : 0x88A4
- Charge utile (*payload*) entre 42 et 1500 octets
- CRC (4 octets)
- *InterPacket Gap* (IPG) sur 12 octets

HOW STANDARDS PROLIFERATE:
(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)



Bus et liaisons: Récapitulatif

Les périphériques de communication à savoir manipuler/configurer

- UART
- SPI
- I2C
- CAN

Les différents modes de communication

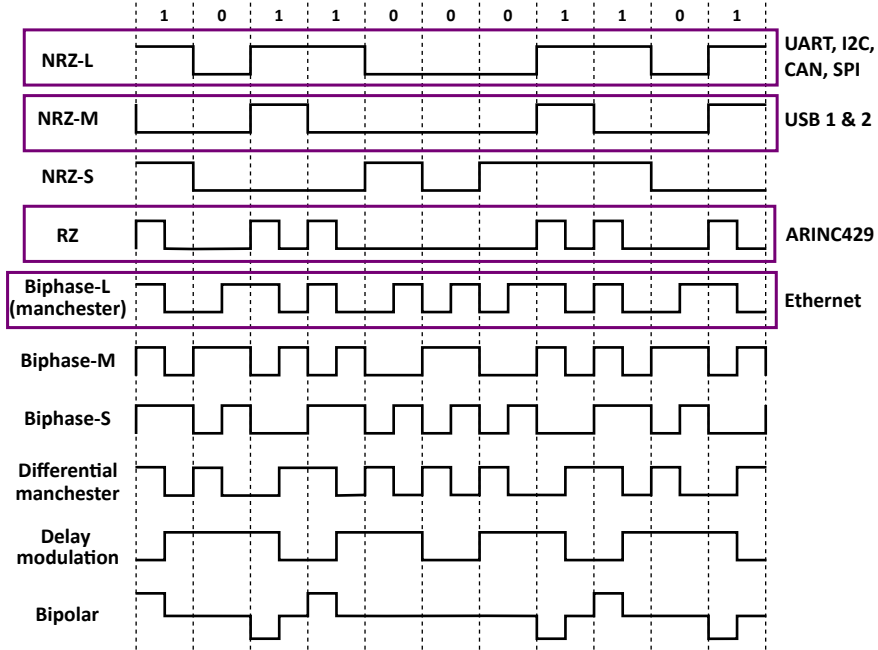
- Point à point
- Bus

Les différents encodages

- NRZ & NRZI
- Manchester

Quel périphérique pour quelle application ?

- Distance
- Débit
- Nombre de composants



- ① Les microcontrôleurs
- ② Spécificités logicielles
- ③ Les périphériques
- ④ Bus et liaisons
- ⑤ Génie logiciel embarqué
Conception d'un système embarqué

Conception d'un système embarqué : Règles générales

Description du problème

- Diagramme des cas d'utilisation
- ① À quoi va servir le système conçu ? Qu'est-ce qu'il est sensé faire ?
 - a Quelles données d'entrées ?
 - b Quelles actions à prendre ?
 - c Quelles données à envoyer et recevoir ?
 - ② Quelles sont les conditions environnementales dans lequel sera plongé le système ?
 - a Physiques : température, hygrométrie, pression, champs EM, radiations, etc.
 - b Mécaniques : vibrations, chocs, vents, etc.
 - c Chimiques : contact avec des éléments chimiques corrosifs, acides, basiques, conducteurs, etc.
 - ③ Contraintes temps-réel

Conception d'un système embarqué : Exemple

Un ingénieur qualité :

- rentre dans un bar
- rampe dans un bar
- vole dans un bar
- saute dans un bar
- danse dans un bar

et il commande :

- une bière
- 2 bières
- 0 bière
- 99 999 bières
- -1 bière
- un lézard dans une bière
- azertyuiop bières

Test complet et réussi :)

Un client rentre dans le bar, demande où sont les toilettes et le bar explose

Conception d'un système embarqué : État de l'art et préconception

- ④ Rechercher les différents moyens ou techniques d'acquisition
 - a Identifier les différents capteurs clé en main (intégrés)
 - b Identifier d'autres grandeurs physiques qui dépendent ou sont images de celle que vous souhaitez mesurer
- ⑤ Rechercher les différents moyens ou techniques d'action
 - a Identifier les différents actionneurs clé en main
 - b Identifier les autres grandeurs physiques qui influent celle que vous souhaitez actionner
- ⑥ Rechercher les différents moyens de communiquer
 - a Identifier les émetteurs et récepteurs des communication
 - b Identifier les débits et fréquences de communication
 - c Identifier les mutualisations possibles (bus)
- ⑦ Rechercher les différents microcontrôleurs pouvant répondre aux besoins identifiés
 - a Estimer le nombre de pins requis, la puissance de calcul et la mémoire nécessaire à l'application
 - b Si besoin est, partitionner les tâches entre plusieurs μC

Conception d'un système embarqué : Outils de conception

- 8 Faire des schémas blocs de l'ensemble du système
 - a Placer les différents blocs
 - b Relier les blocs en indiquant le type de liaison
- 9 Réaliser les diagrammes d'activité et d'états-transitions
 - a Lister les tâches/activités
 - b Ordonner les tâches en fonction des dépendances
 - c Identifier les conflits au niveau matériel
- 10 Faire l'implémentation
 - a Réaliser les schémas électriques
 - b Écrire le code en respectant les diagrammes précédents
- 11 Vérifier l'implémentation
 - a Faire des tests unitaires 
 - b Débuguer les interfaces à l'oscilloscope ou analyseur logique